

Skip List

William Pugh, 1989



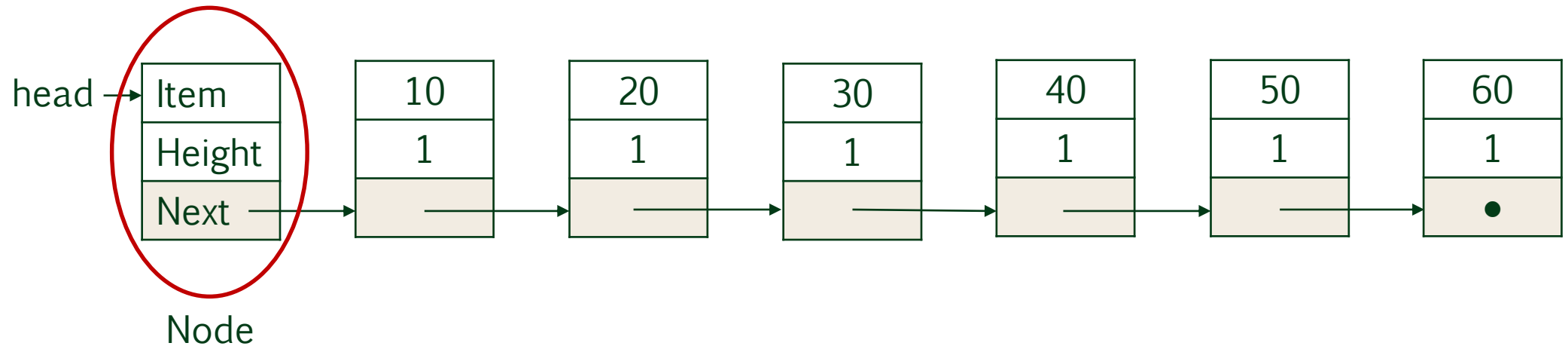


Background

- › A skip list is a randomly built binary search tree
- › The expected time complexity to insert and remove an item from the skip list is $O(\log n)$
- › Inspired as a generalization of the singly-linked list



Consider the following linked list

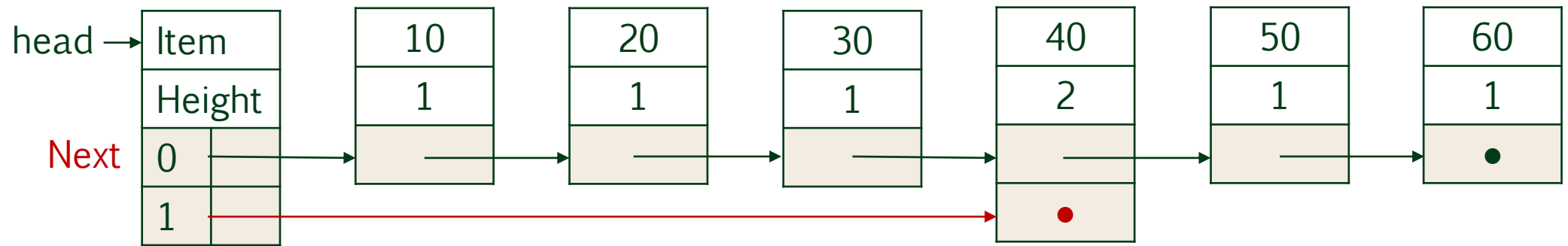


In the worst case, it takes $O(n)$ time to:

- 1) insert an item,
- 2) find an item, and
- 3) remove an item

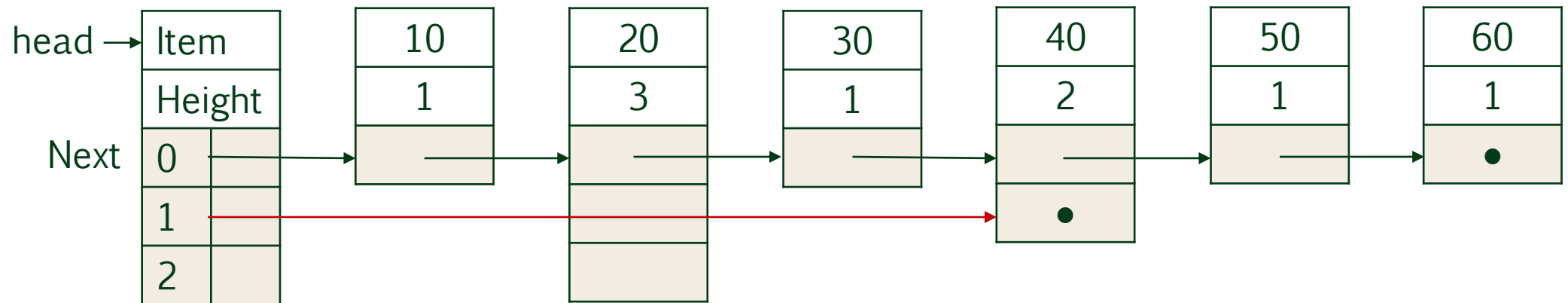


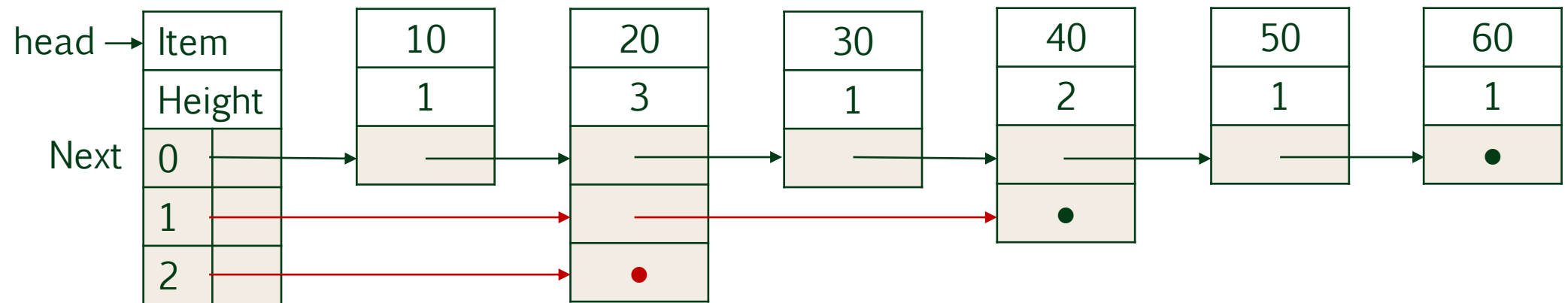
What if we added a level to node 40?





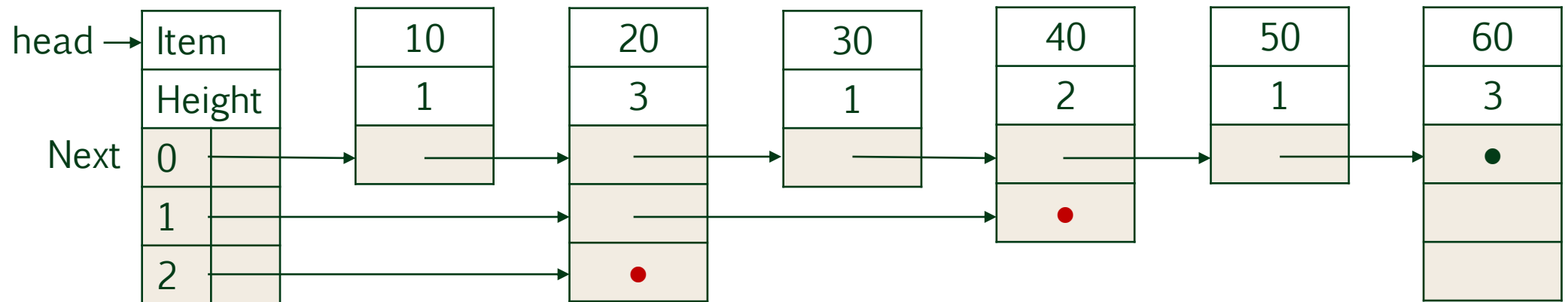
What if we added two levels to node 20?

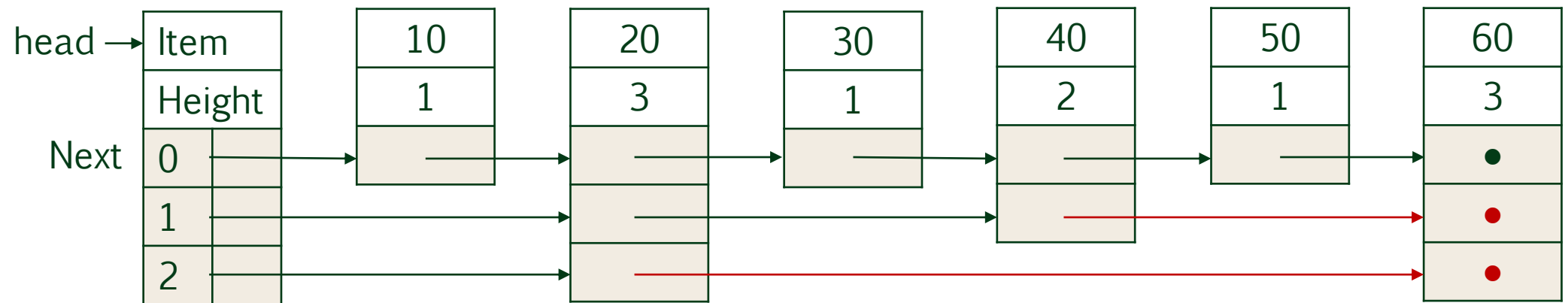






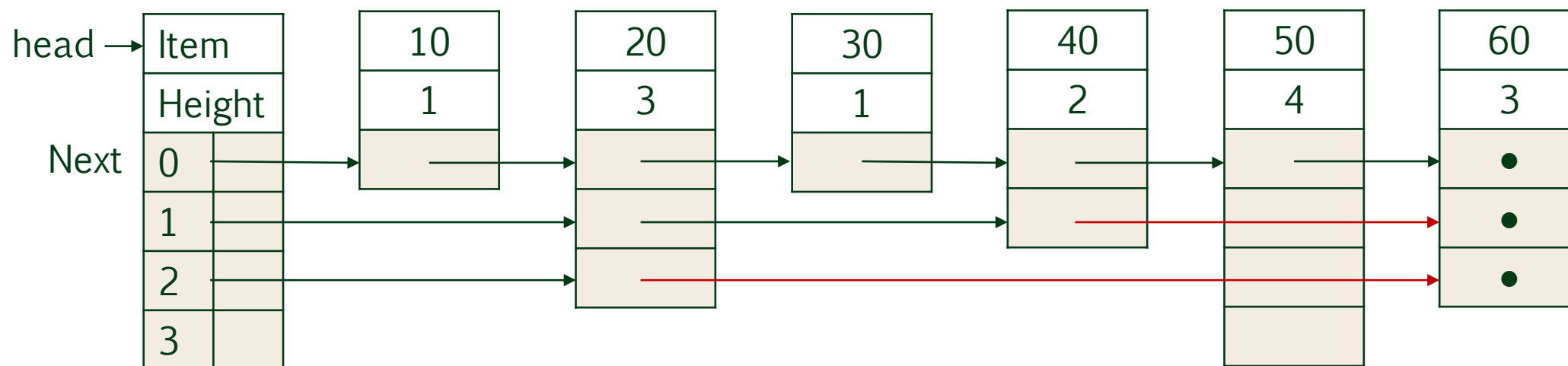
What if we added two levels to node 60?

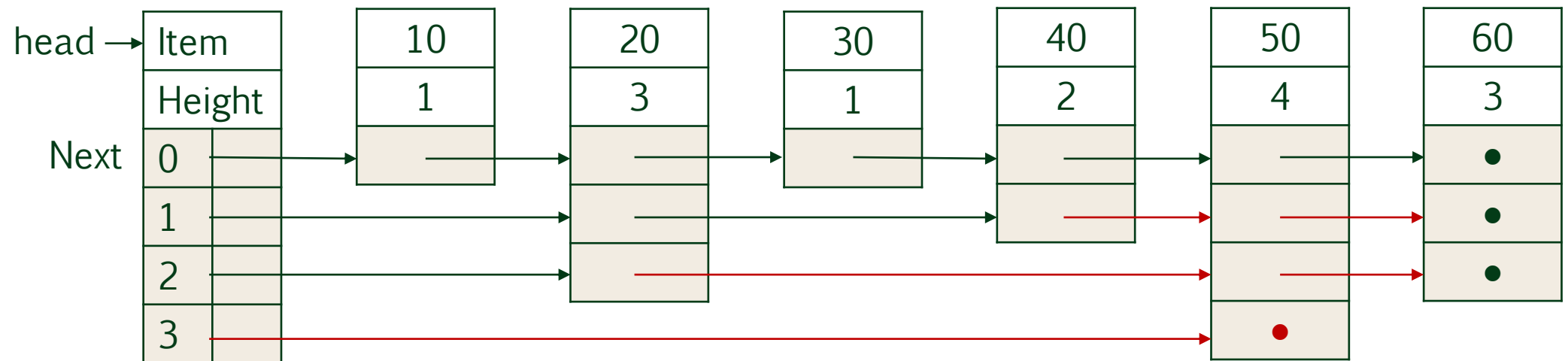






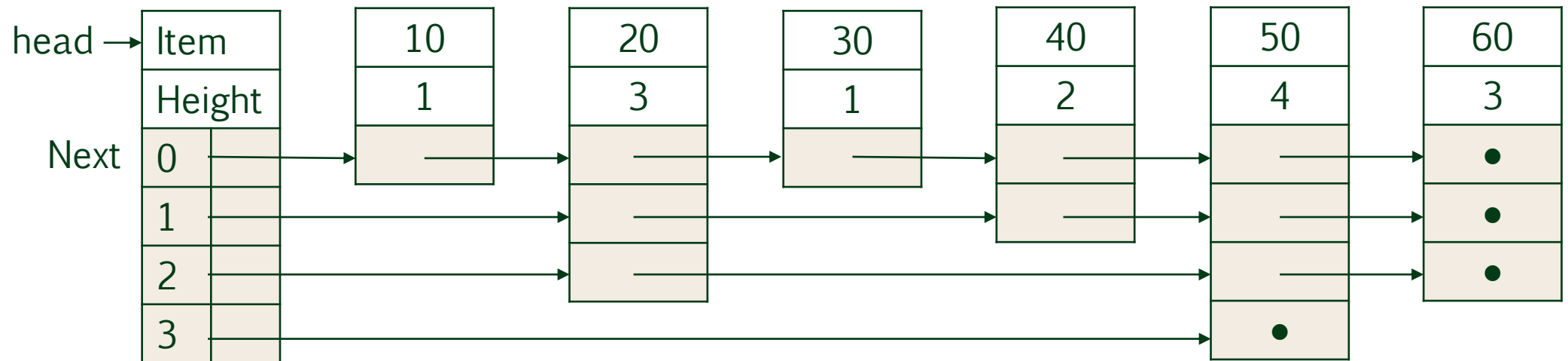
What if we added three levels to node 50?





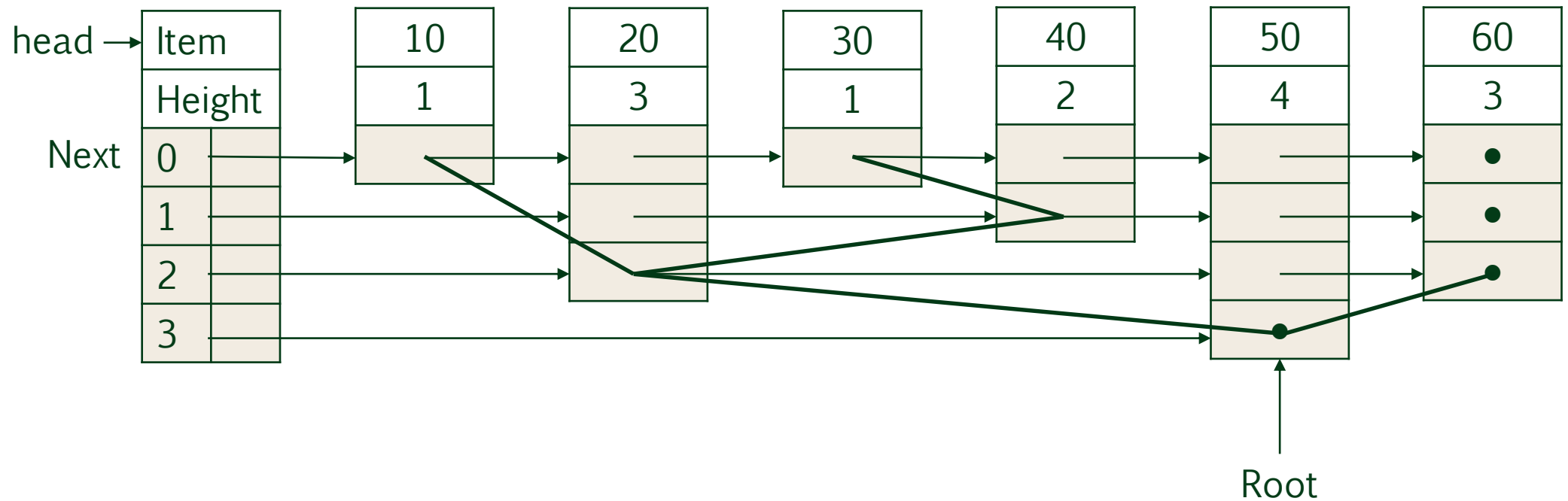


Can you see the binary search tree?





Can you see the binary search tree?



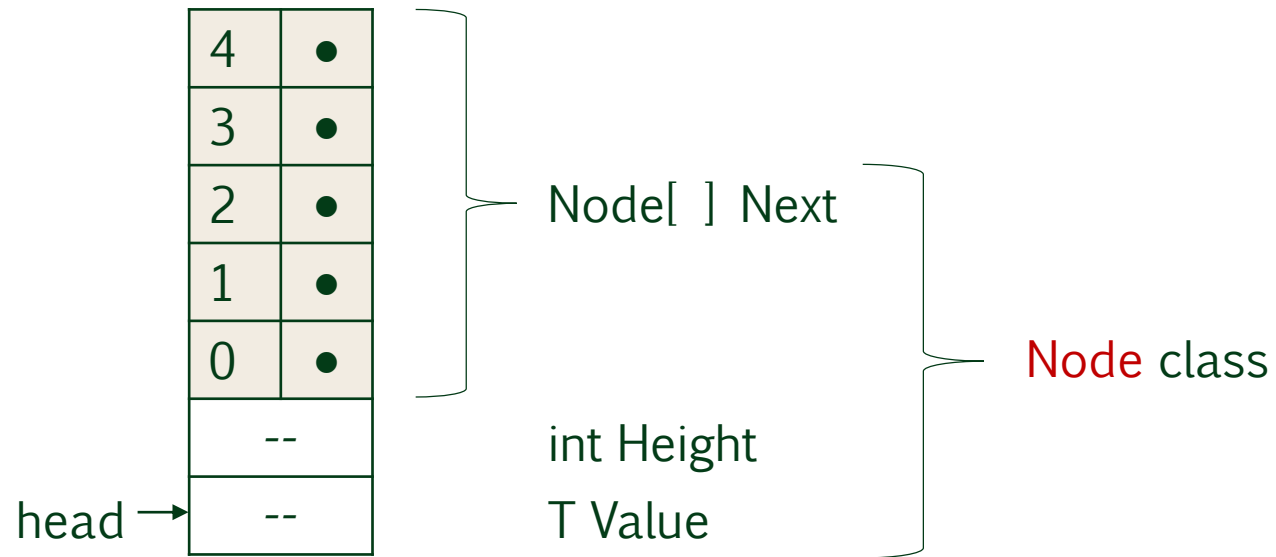


Methods

- › void Insert (T item)
 - inserts item into the skip list (duplicate items are permitted)
- › bool Contains (T item)
 - Returns true if item is found; false otherwise
- › void Remove (T item)
 - Removes one occurrence of item (if possible)



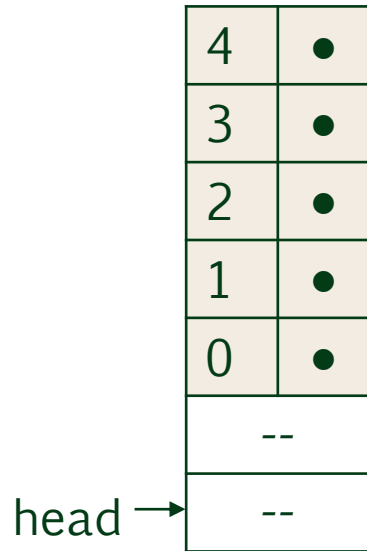
Starting from scratch



`maxHeight = 0 // current maximum height of the skip list`



Insert 30

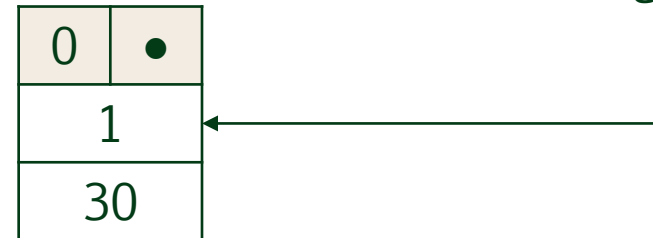


maxHeight = 0

The height of a node is randomly set according to the following distribution

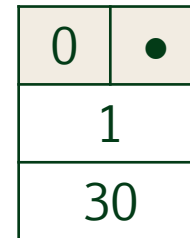
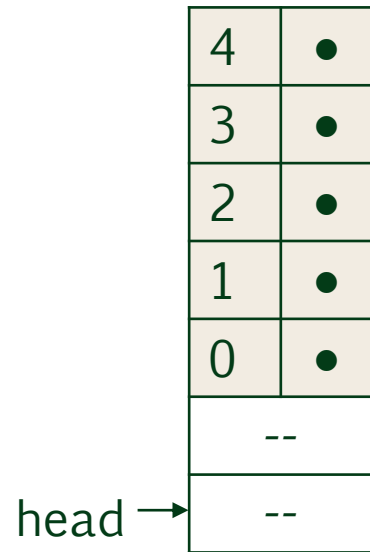
Height	Probability
1	1/2
2	1/4
3	1/8
4	1/16
h	1/2 ^h

but is capped at one more than the current maxHeight of the skip list





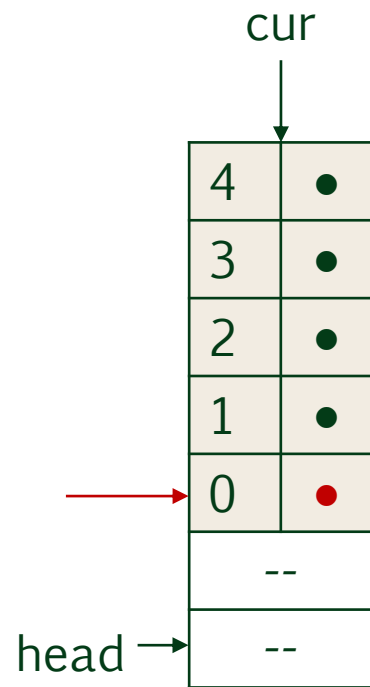
Insert 30



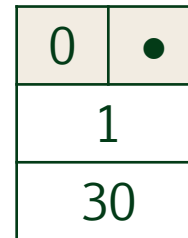
maxHeight = 1



Insert 30



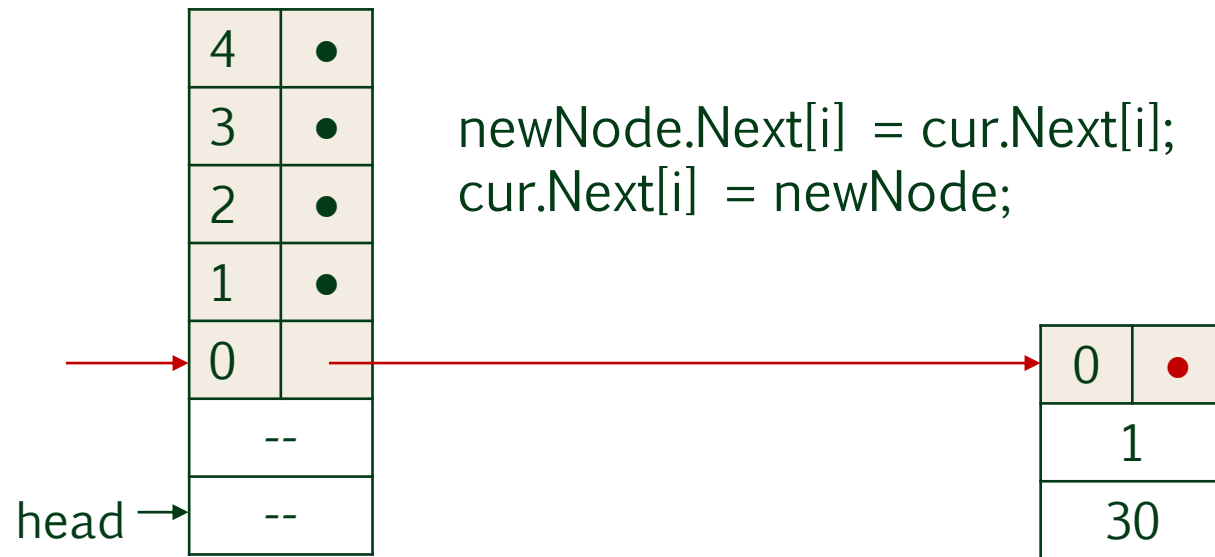
`newNode.Next[i] = cur.Next[i];`
`cur.Next[i] = newNode;`



`maxHeight = 1`



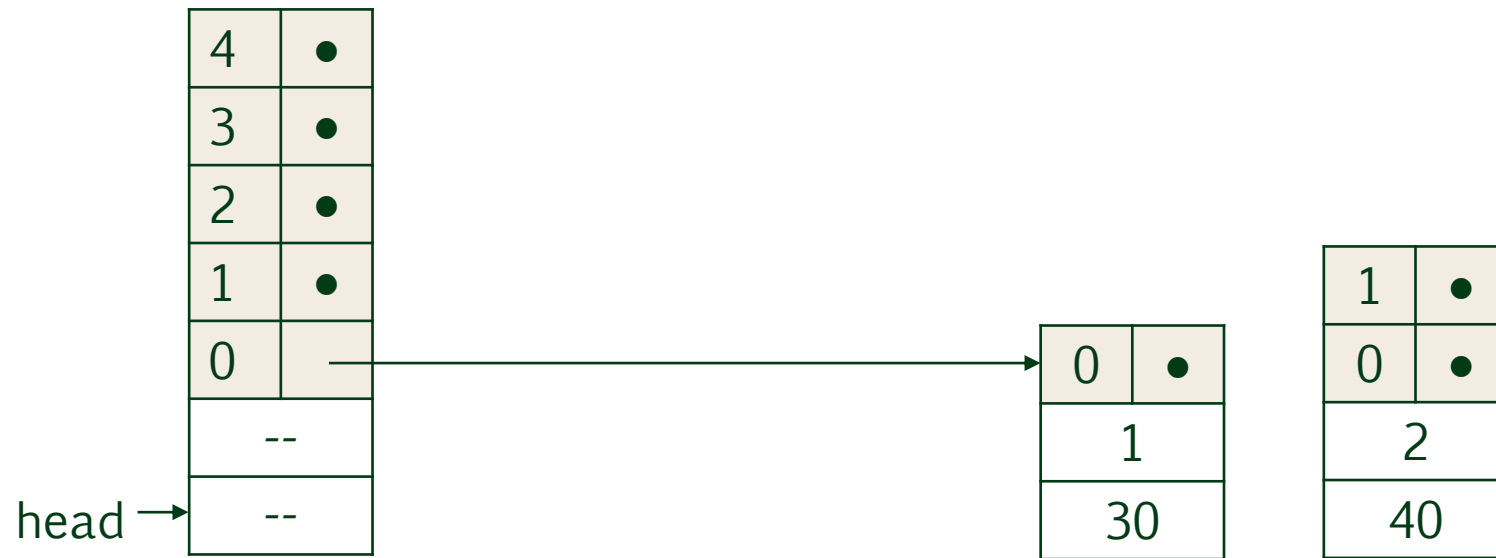
Insert 30



maxHeight = 1



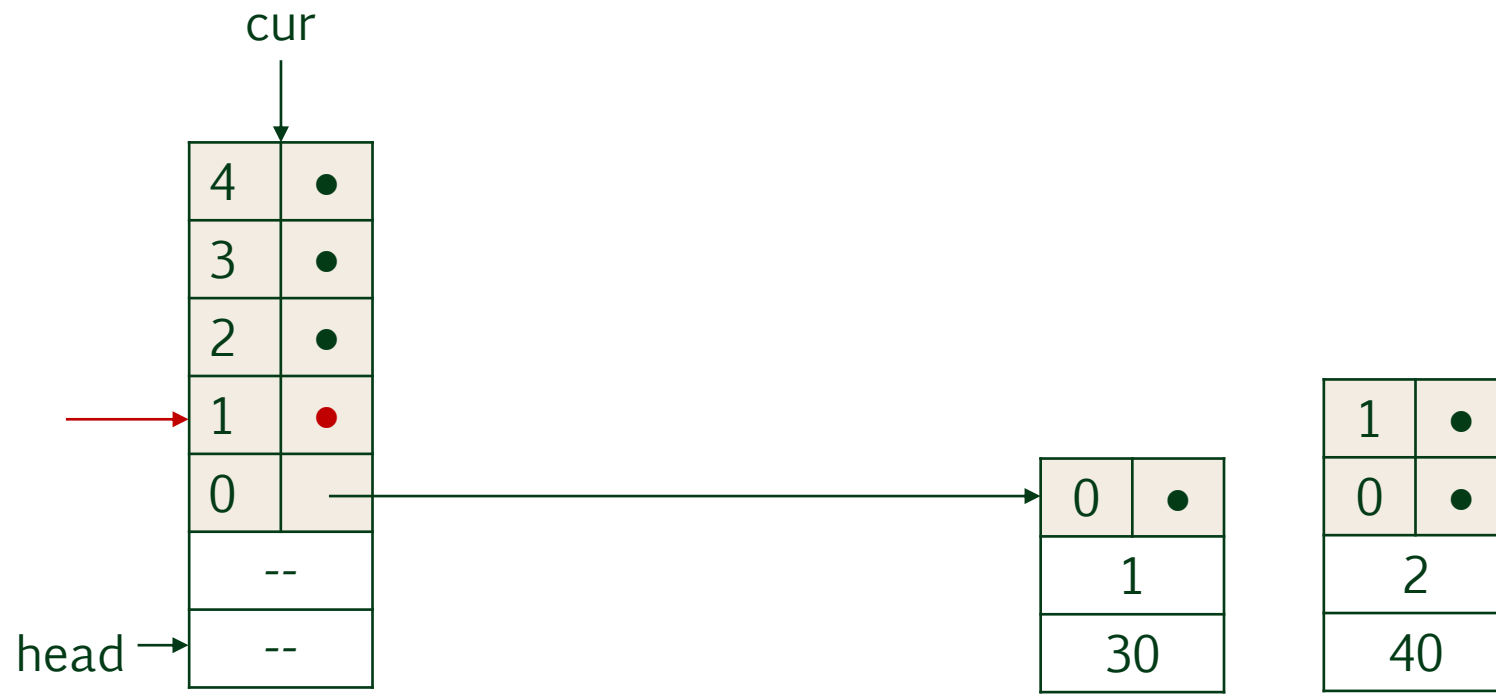
Insert 40



maxHeight = 2



Insert 40

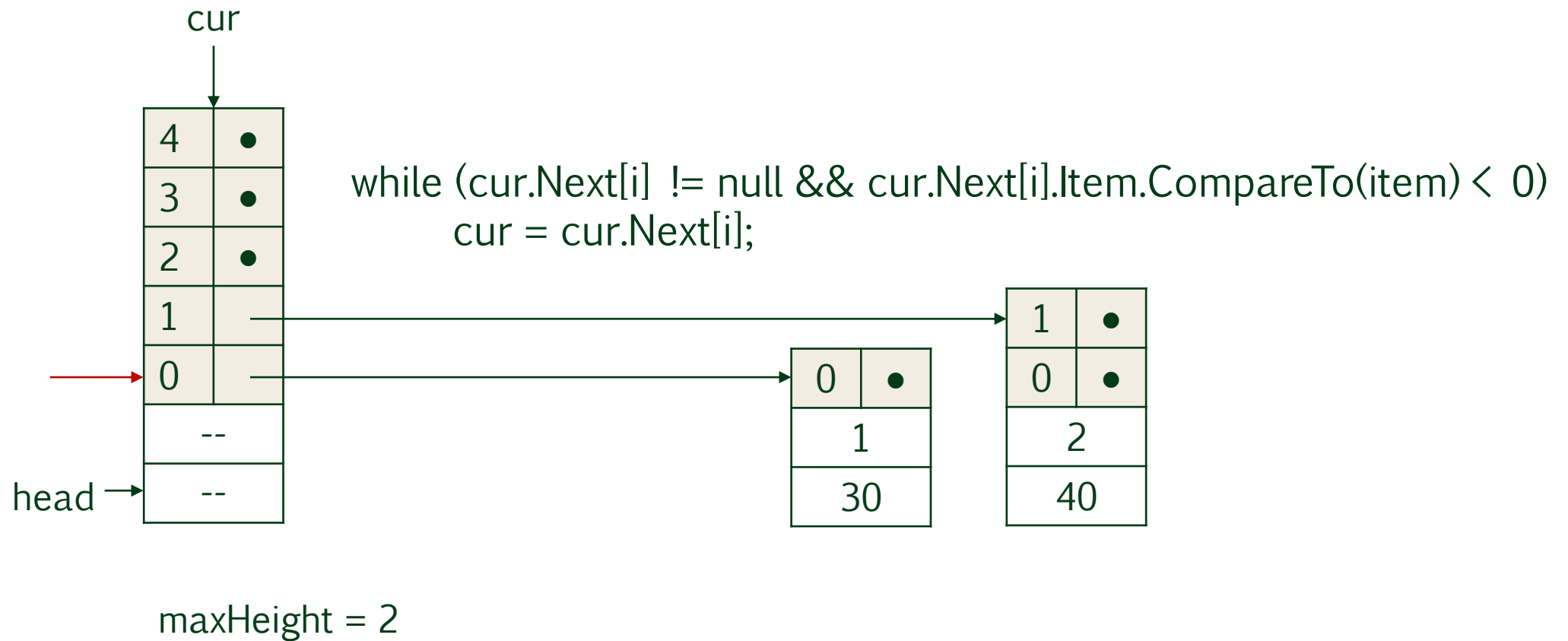


maxHeight = 2



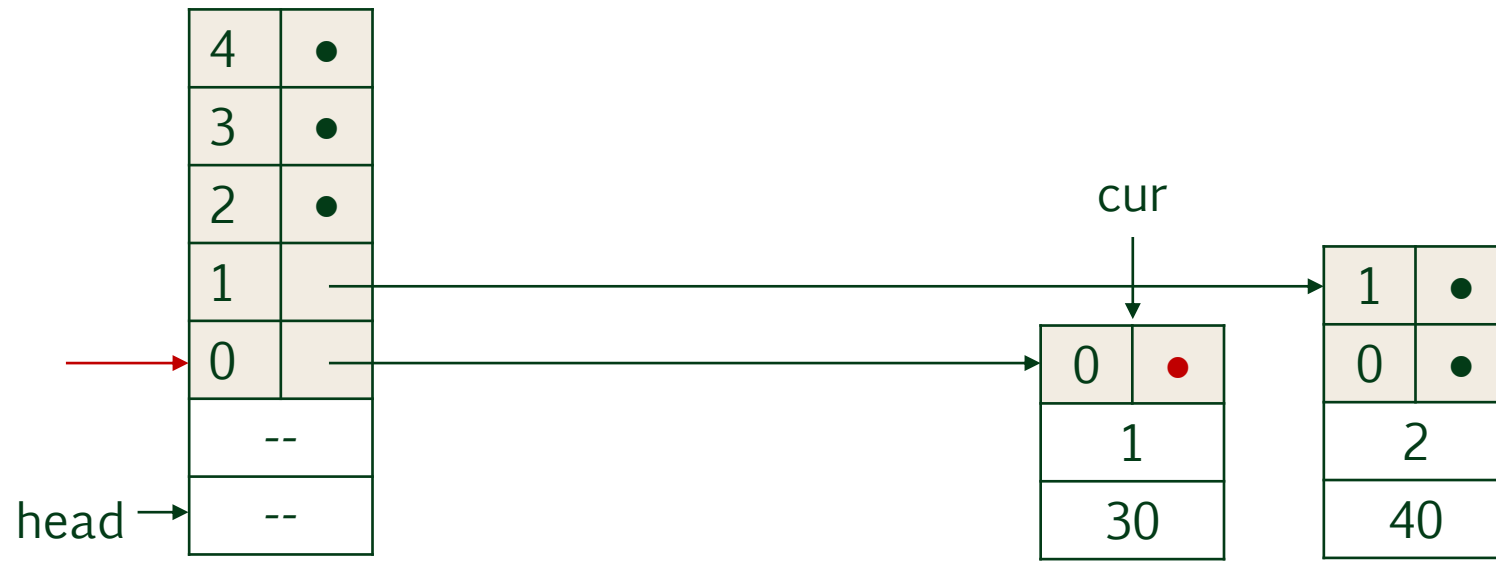


Insert 40





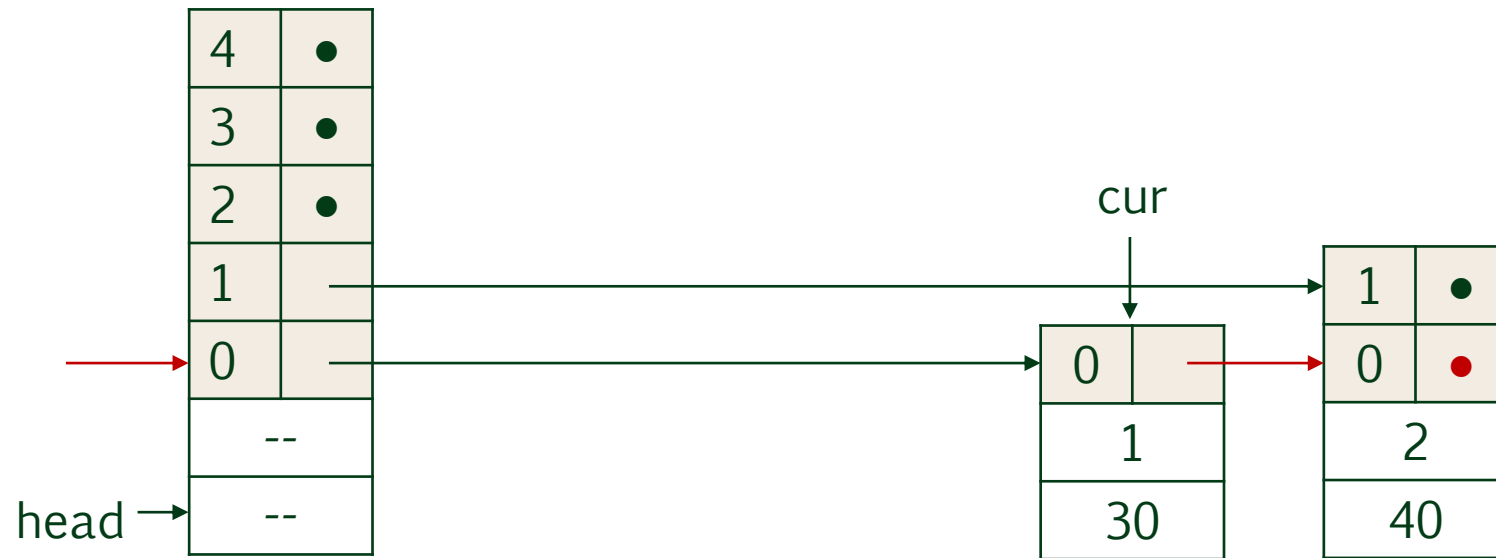
Insert 40



maxHeight = 2



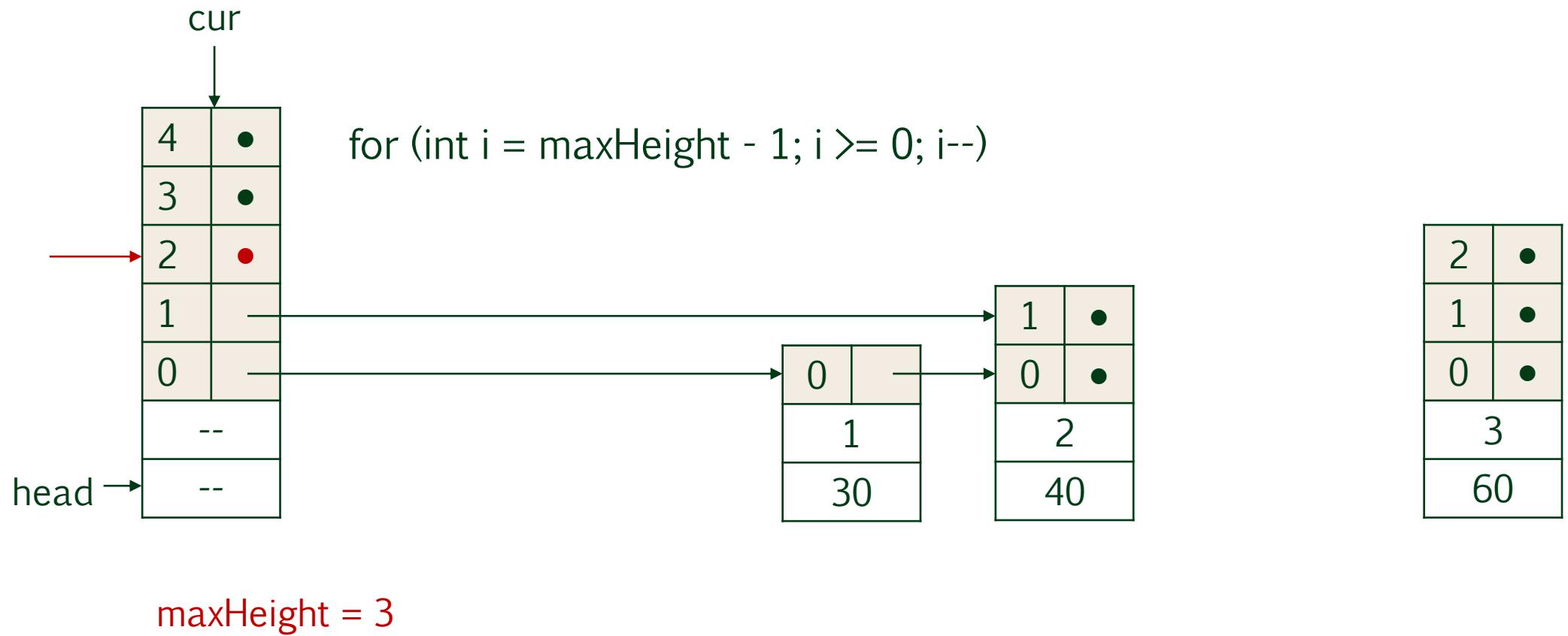
Insert 40



maxHeight = 2

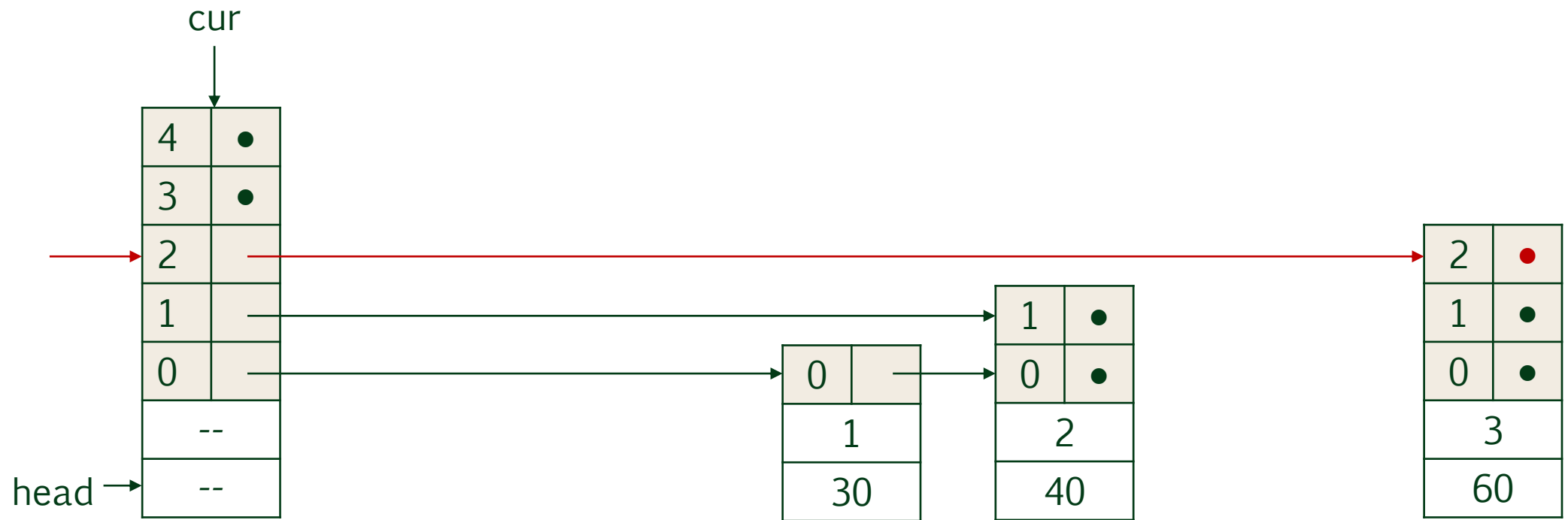


Insert 60





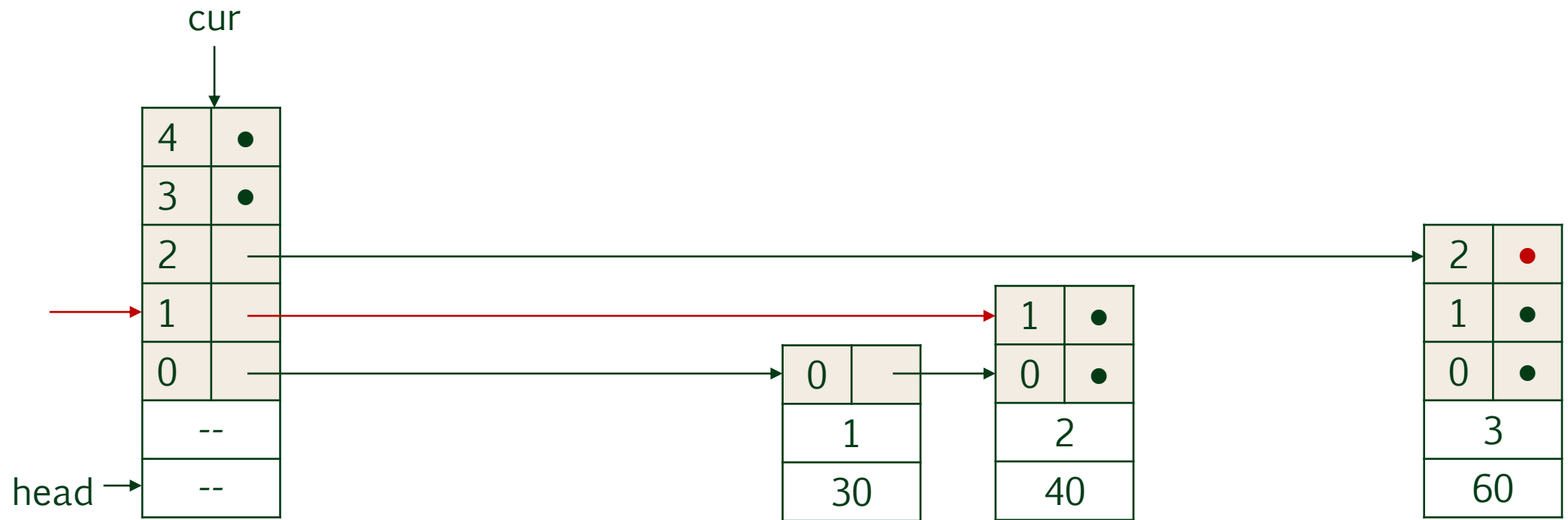
Insert 60



maxHeight = 3



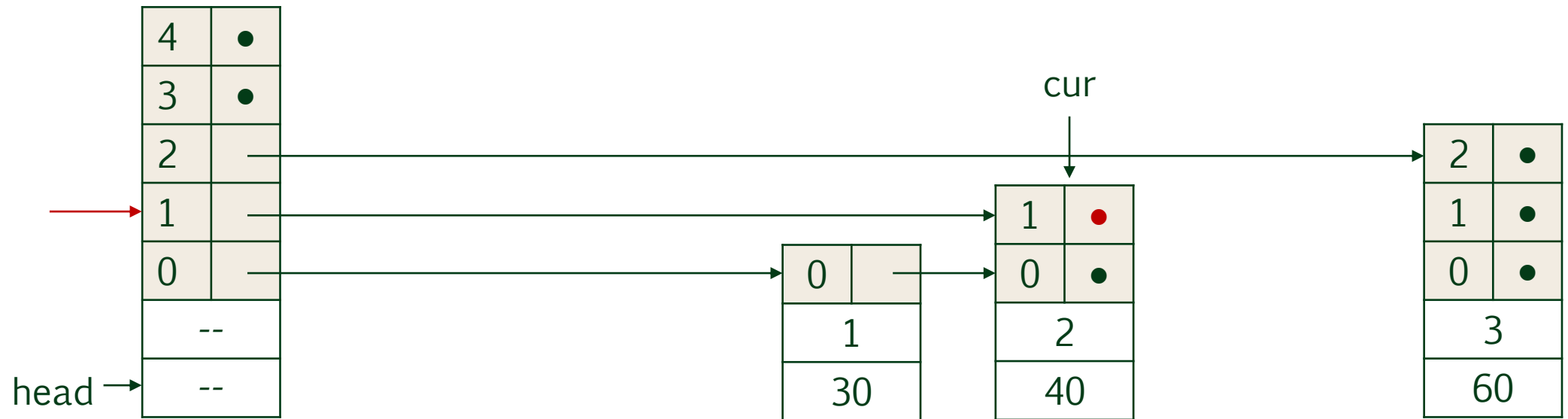
Insert 60



maxHeight = 3



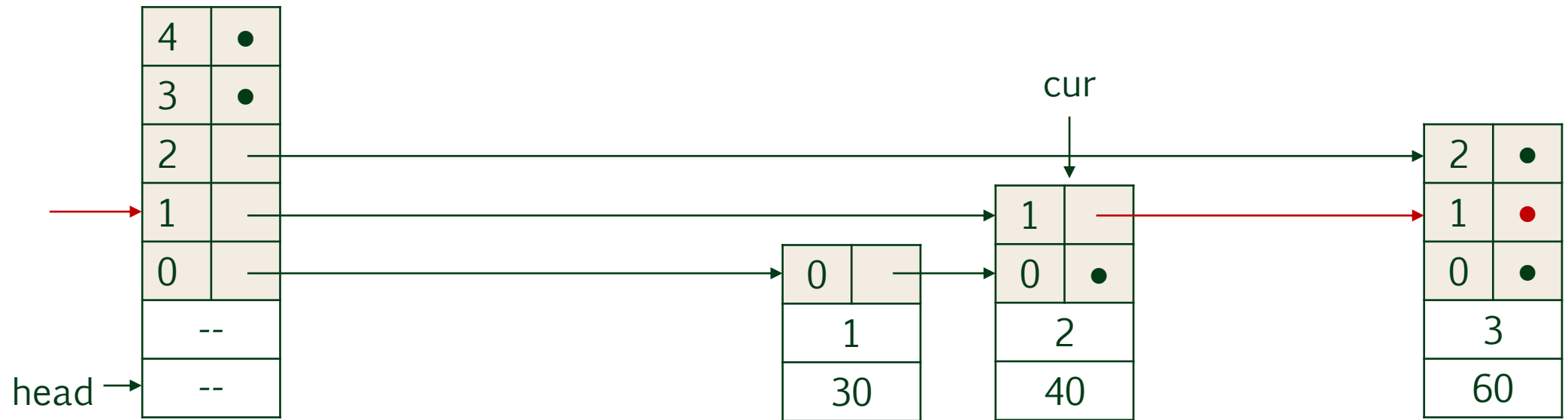
Insert 60



maxHeight = 3



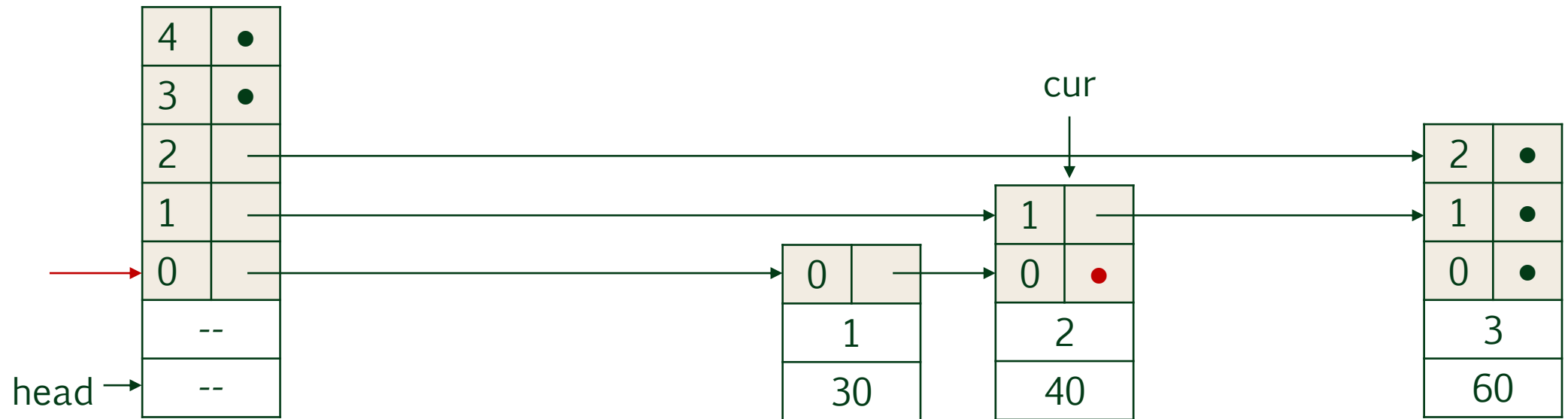
Insert 60



maxHeight = 3



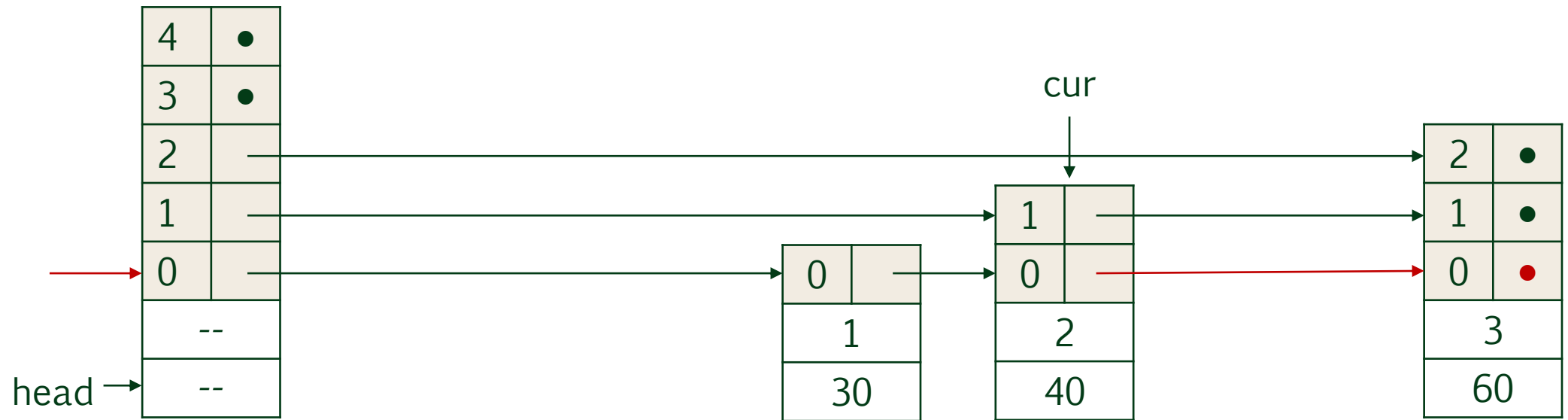
Insert 60



maxHeight = 3



Insert 60

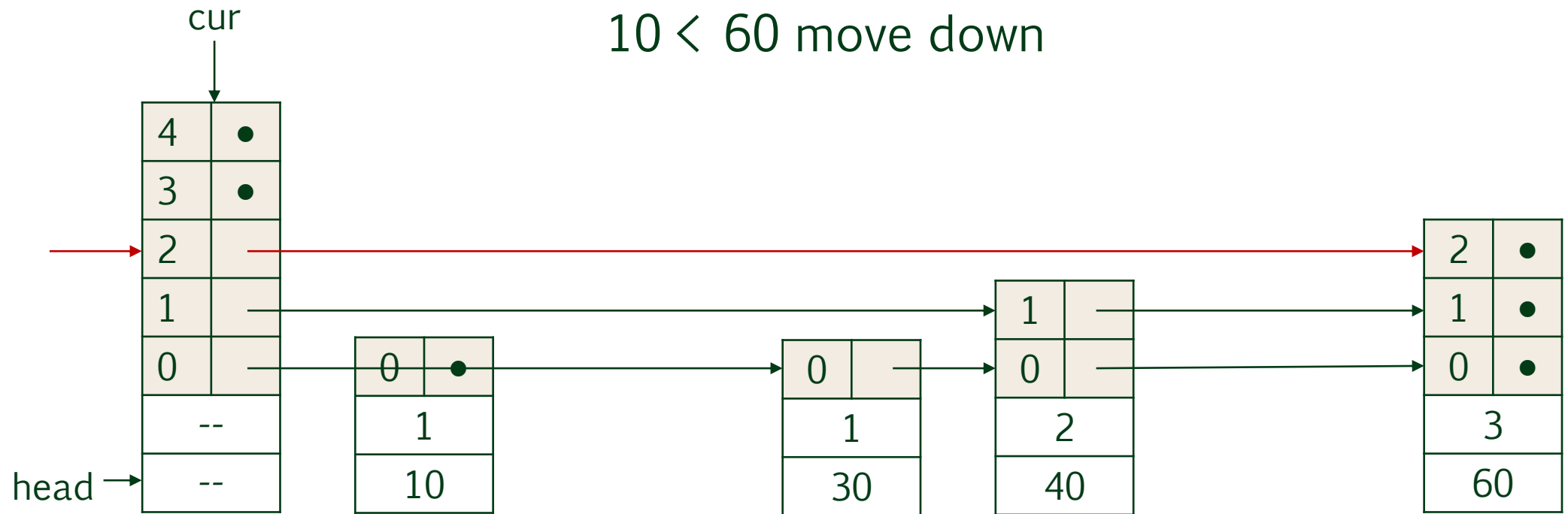


maxHeight = 3



Insert 10

$10 < 60$ move down

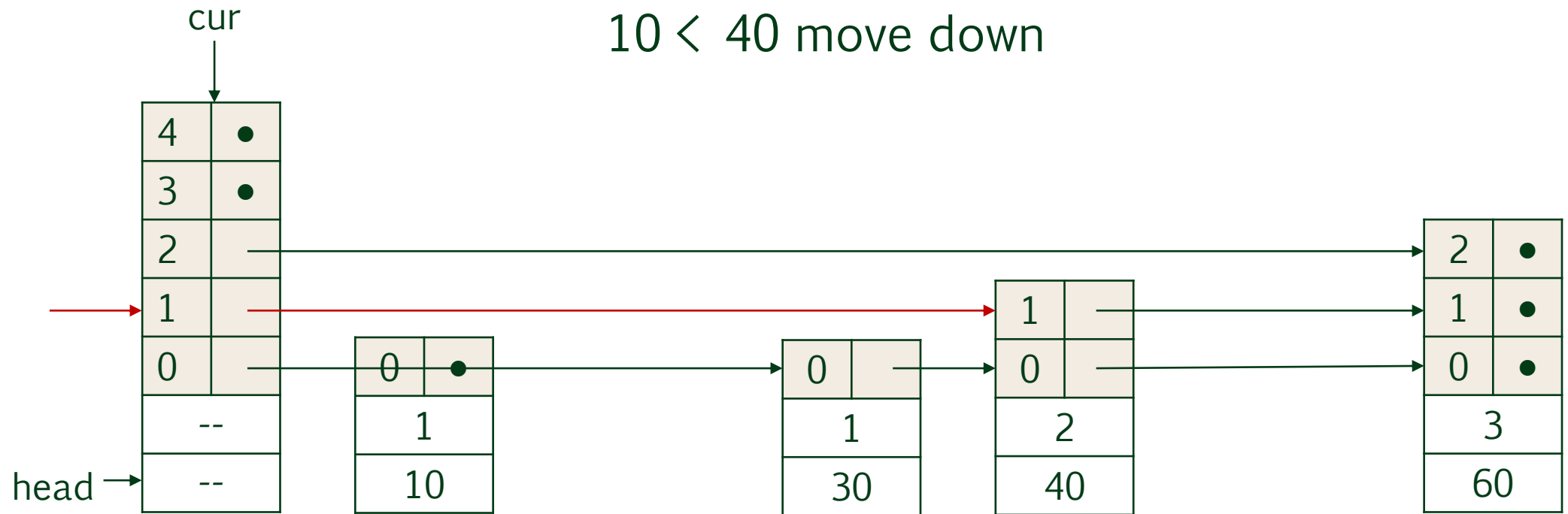


maxHeight = 3



Insert 10

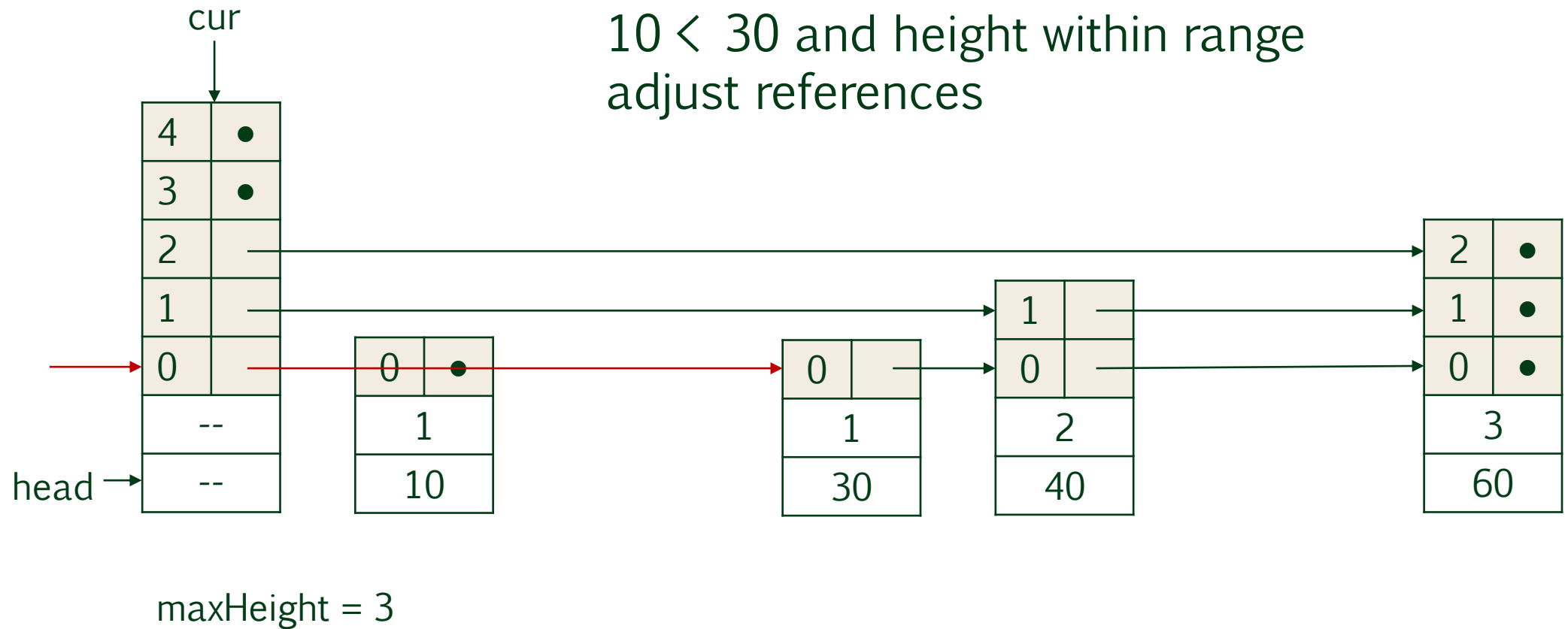
$10 < 40$ move down



maxHeight = 3

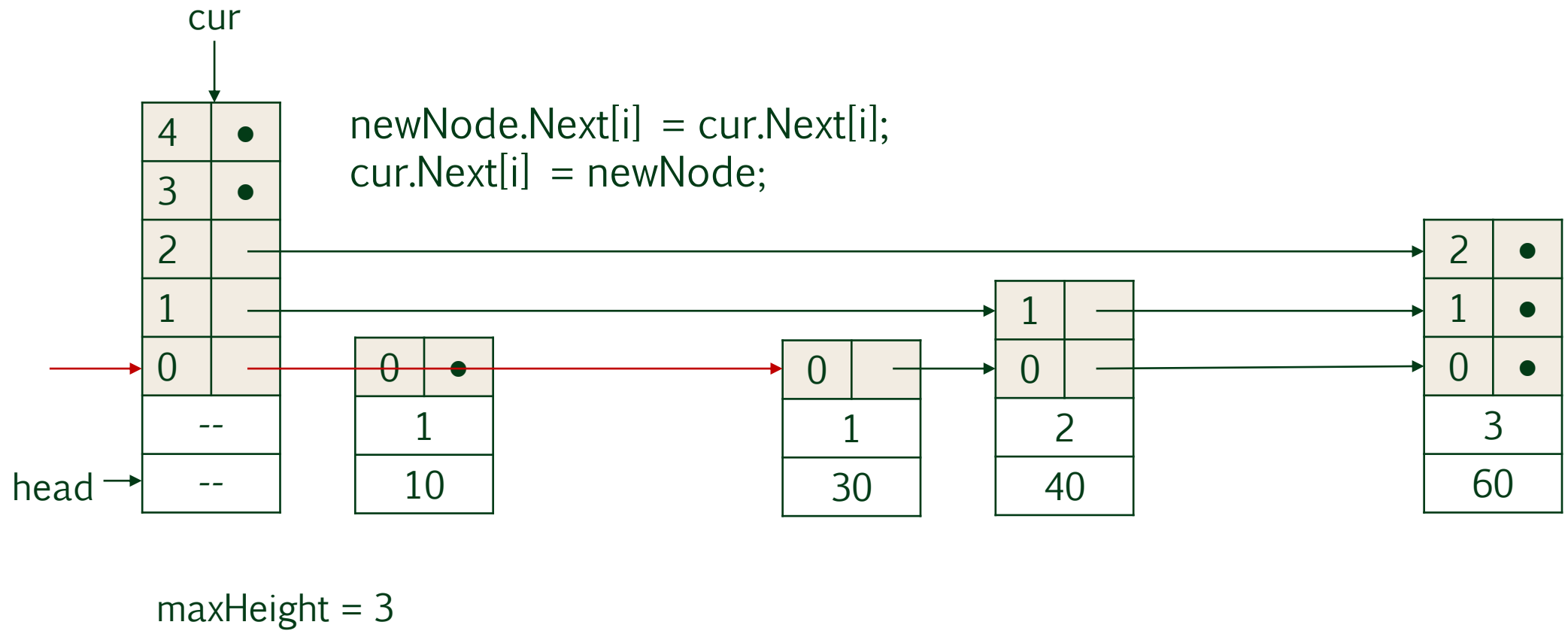


Insert 10



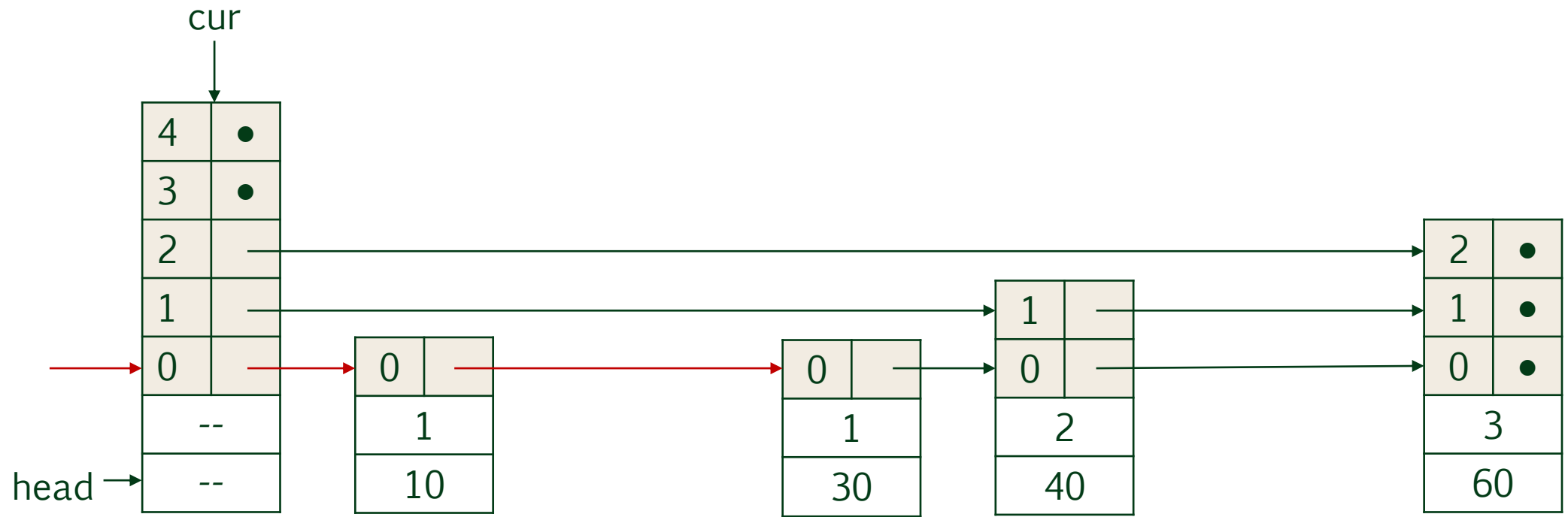


Insert 10





Insert 10

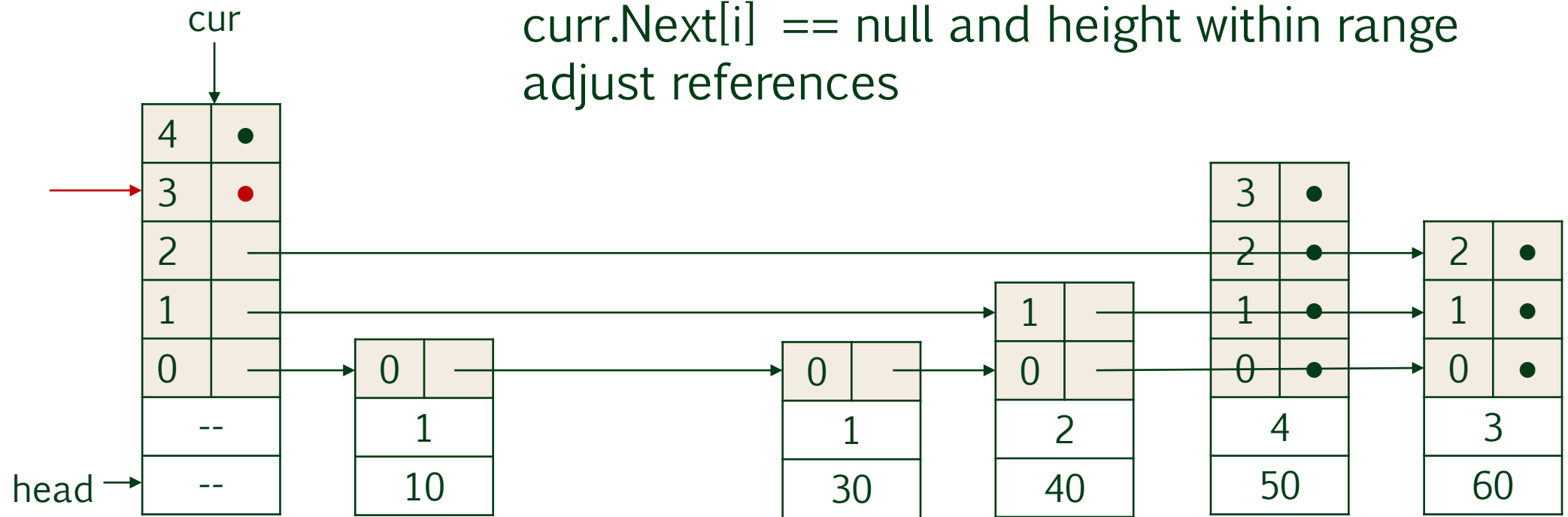


maxHeight = 3



Insert 50

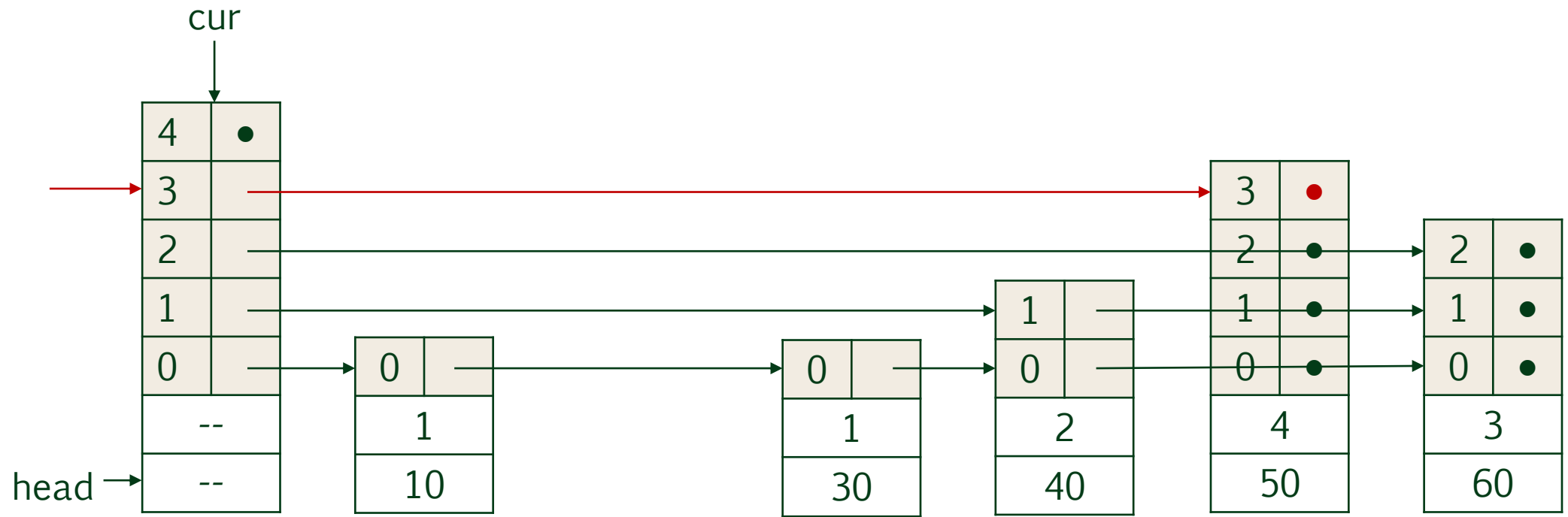
`curr.Next[i] == null` and height within range
adjust references



maxHeight = 4



Insert 50

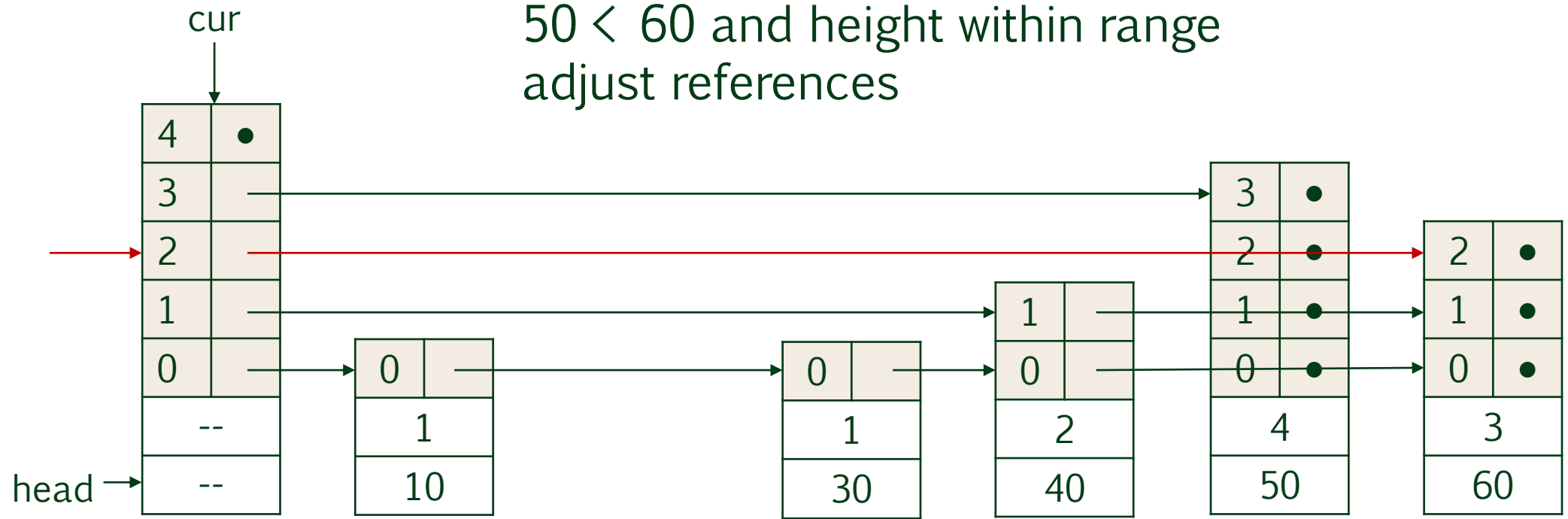


maxHeight = 4



Insert 50

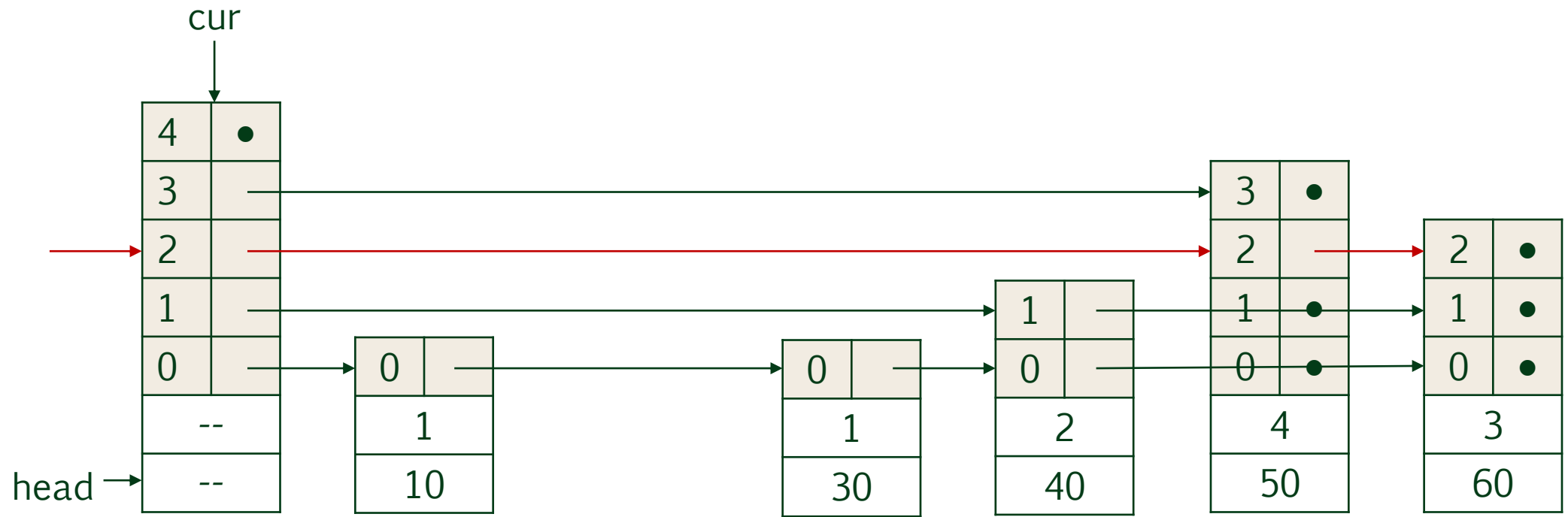
$50 < 60$ and height within range
adjust references



maxHeight = 4



Insert 50

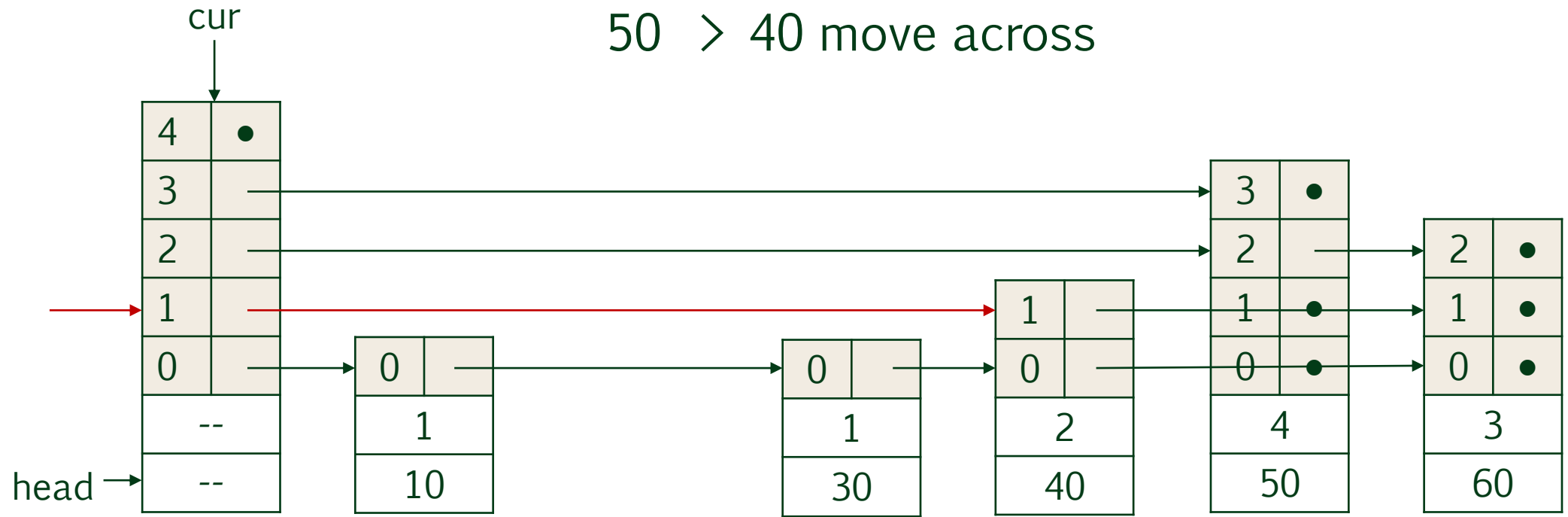


maxHeight = 4



Insert 50

50 > 40 move across

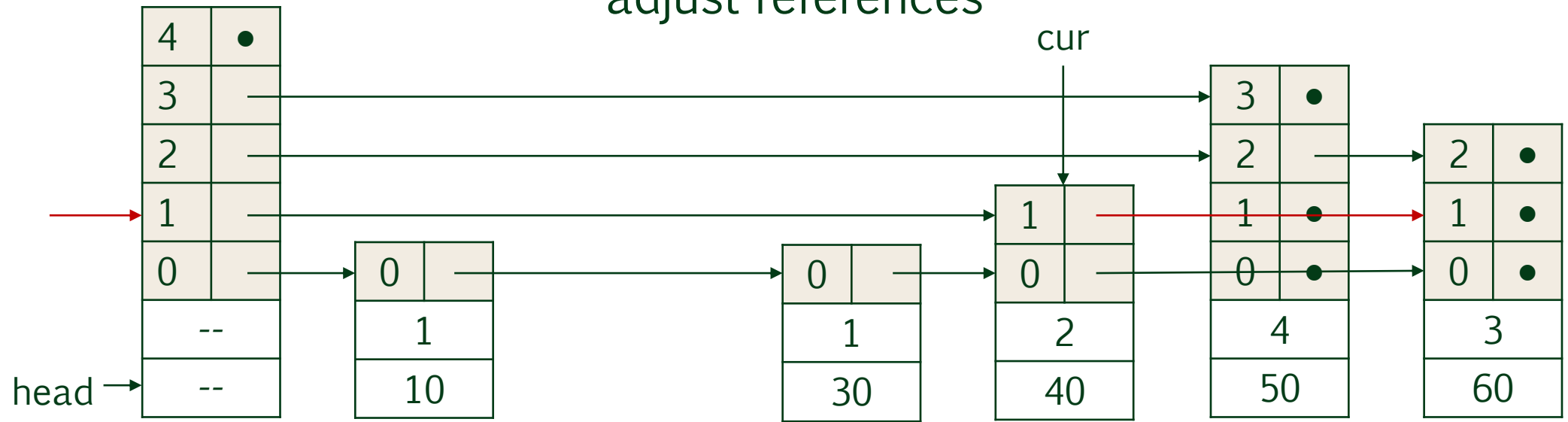


maxHeight = 4



Insert 50

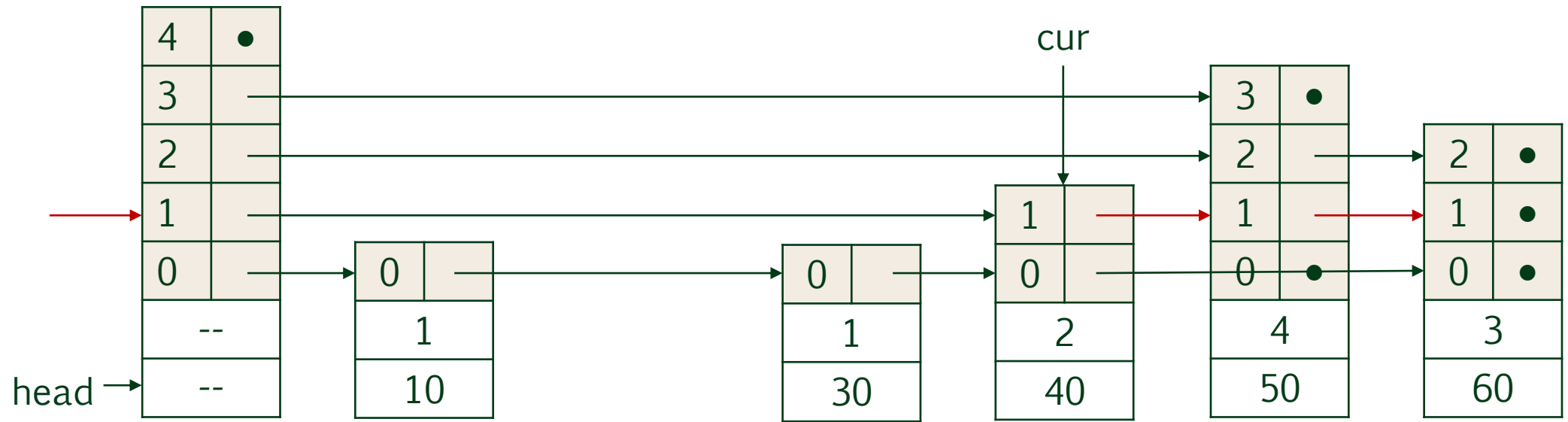
$50 < 60$ and height within range
adjust references



maxHeight = 4



Insert 50

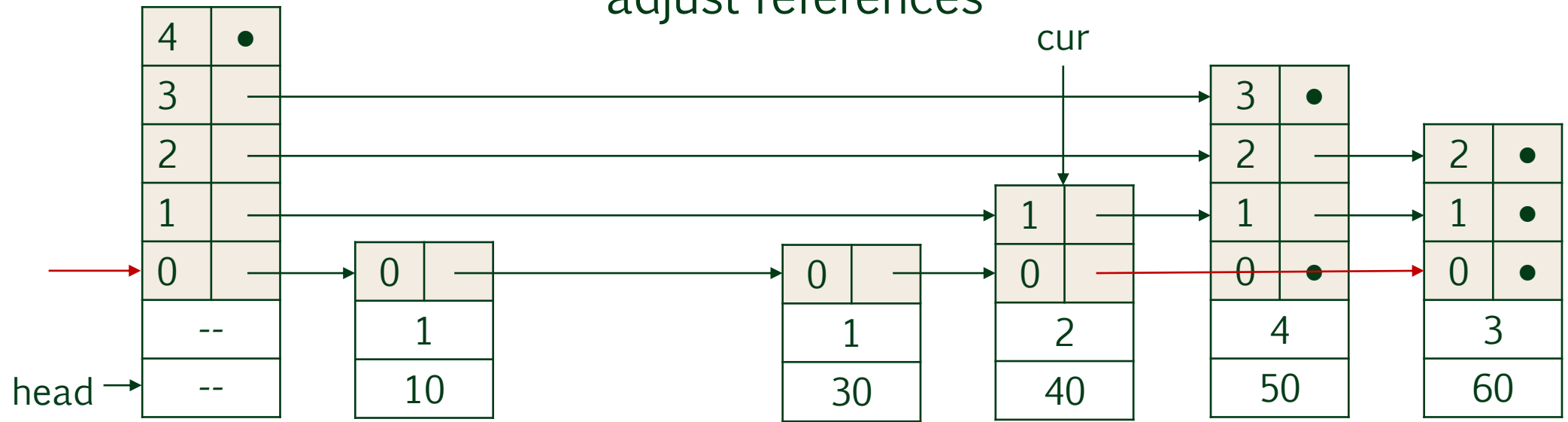


maxHeight = 4



Insert 50

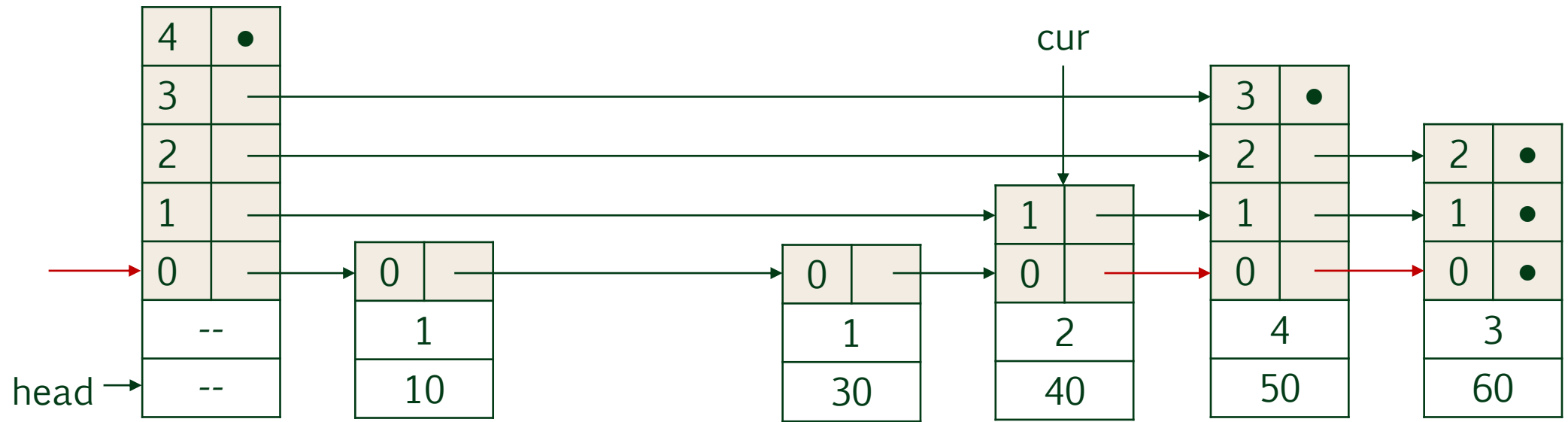
$50 < 60$ and height within range
adjust references



maxHeight = 4



Insert 50

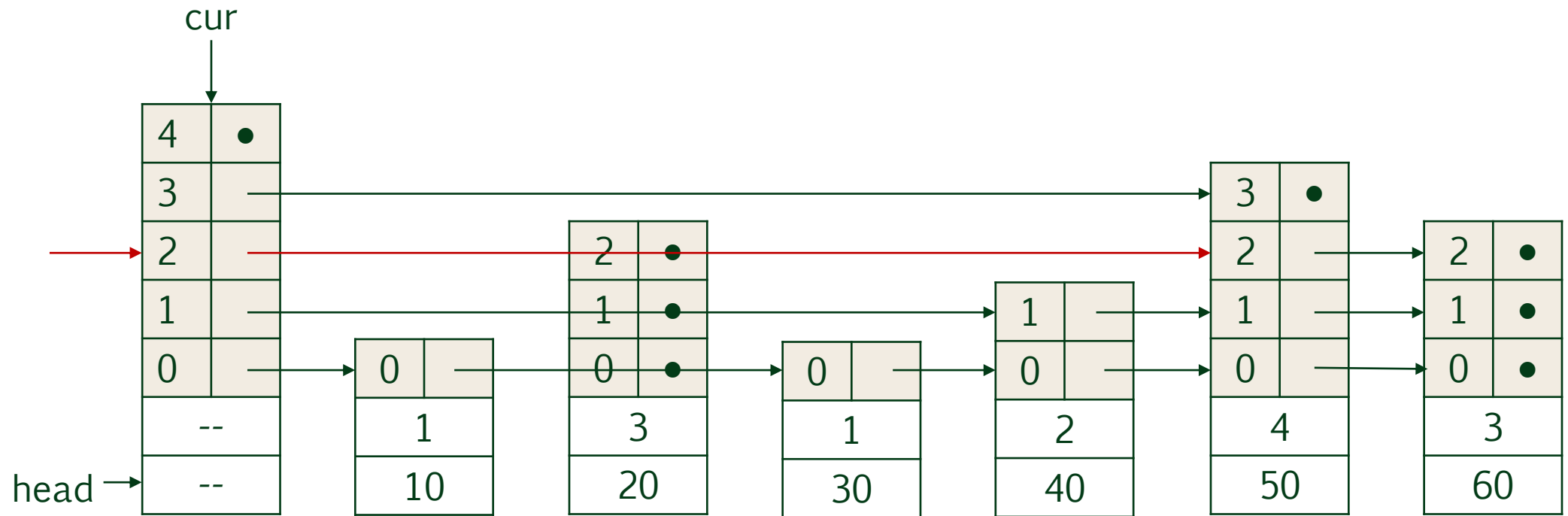


maxHeight = 4





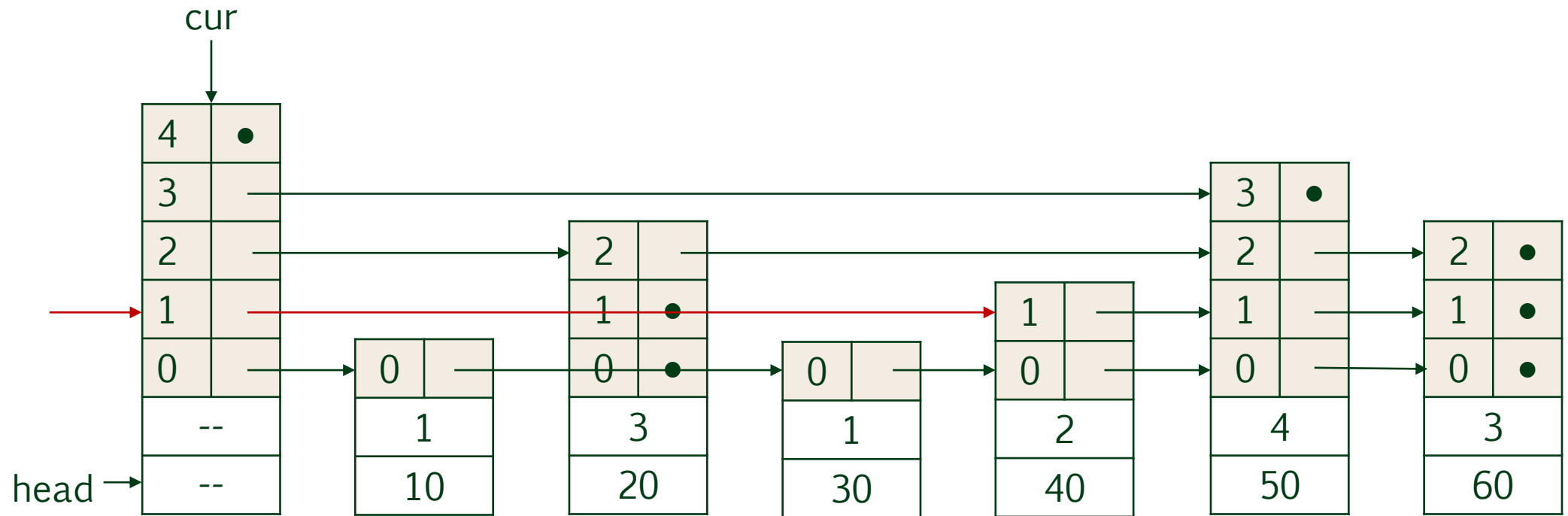
Insert 20







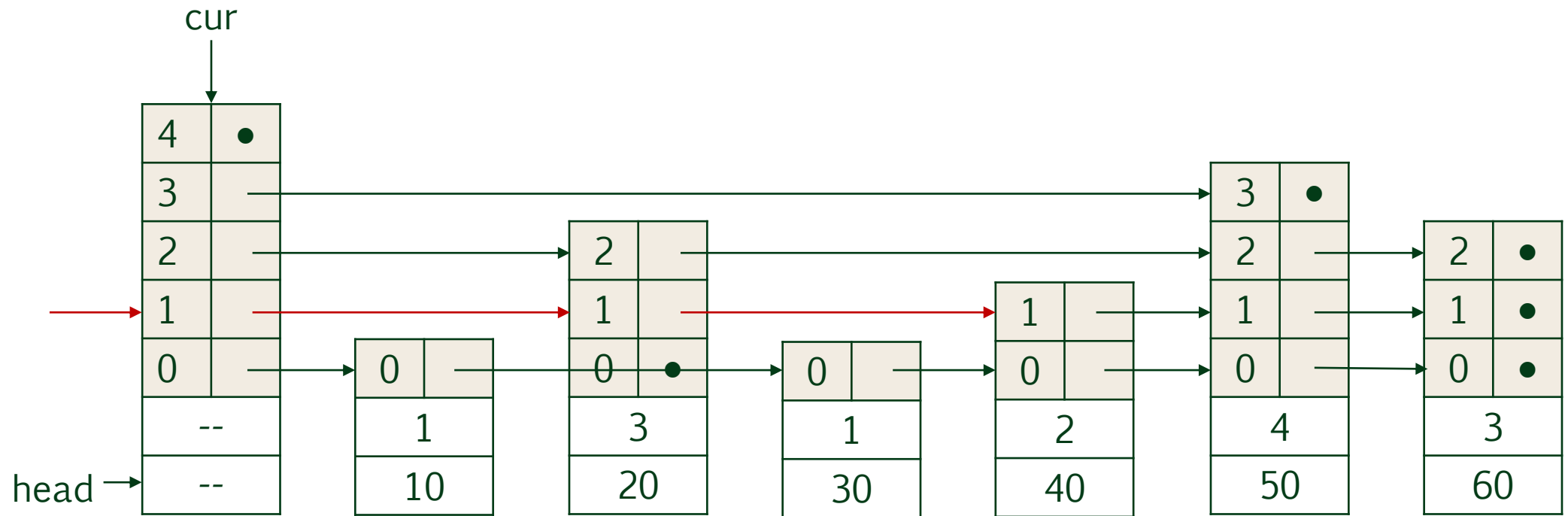
Insert 20



maxHeight = 4



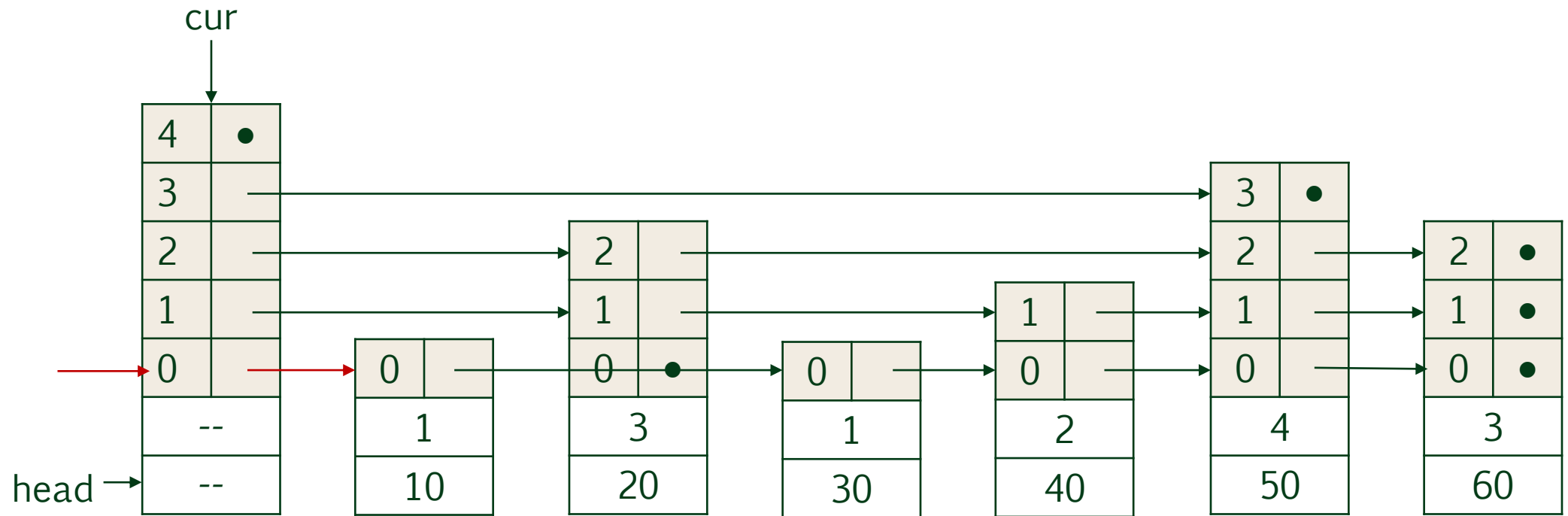
Insert 20



maxHeight = 4



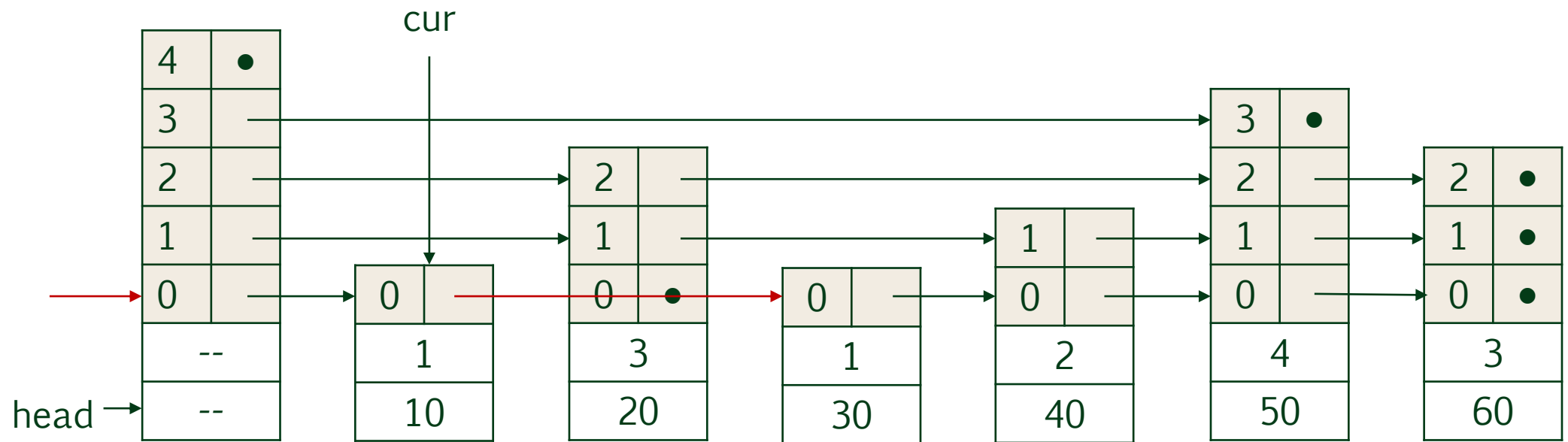
Insert 20



maxHeight = 4



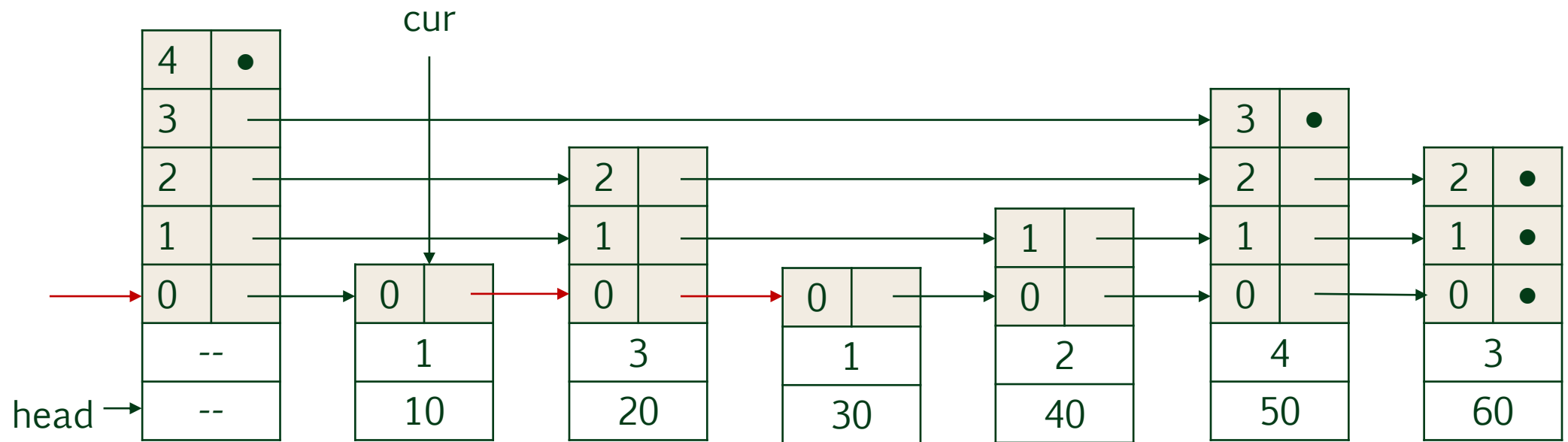
Insert 20



maxHeight = 4



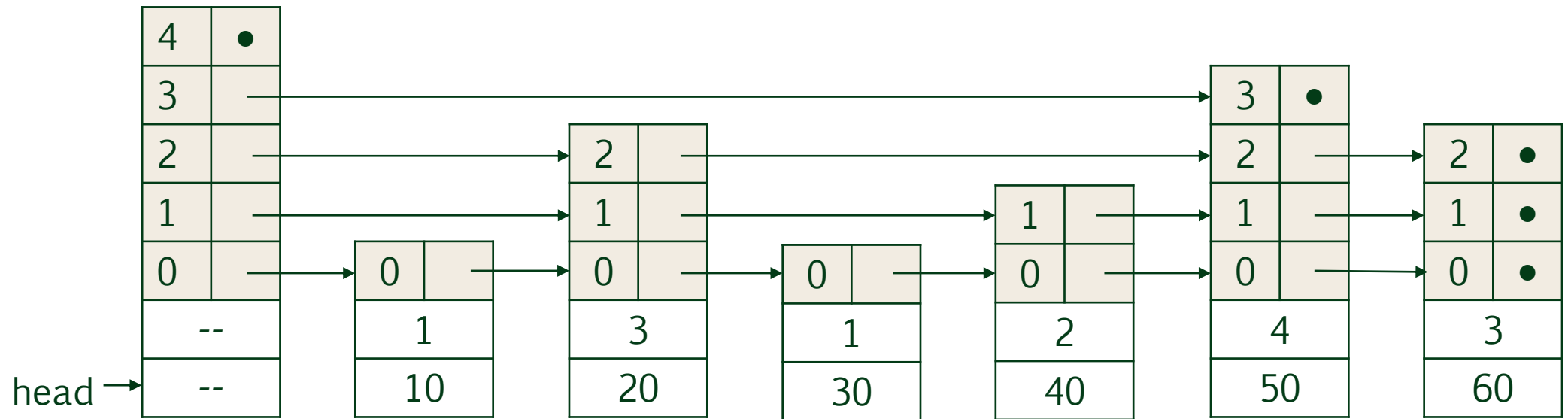
Insert 20



maxHeight = 4



Final Skip List



maxHeight = 4



Phew!



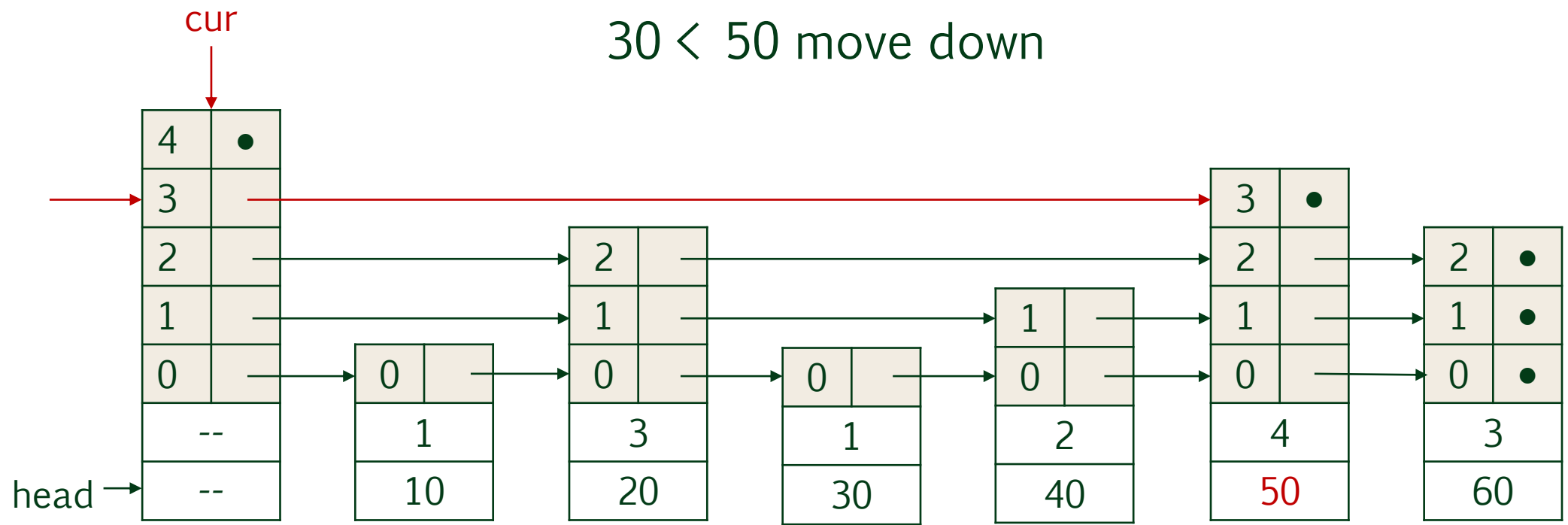


Exercises (using the Final Skip List)

- › Insert 35 with a height of 4.
- › Insert 45 with a height of 1.
- › Insert 40 with a height of 3 (a duplicated item).
- › What is the maximum height of the next insertion?
- › What is the probability that the height of the next node is 5? Be careful.

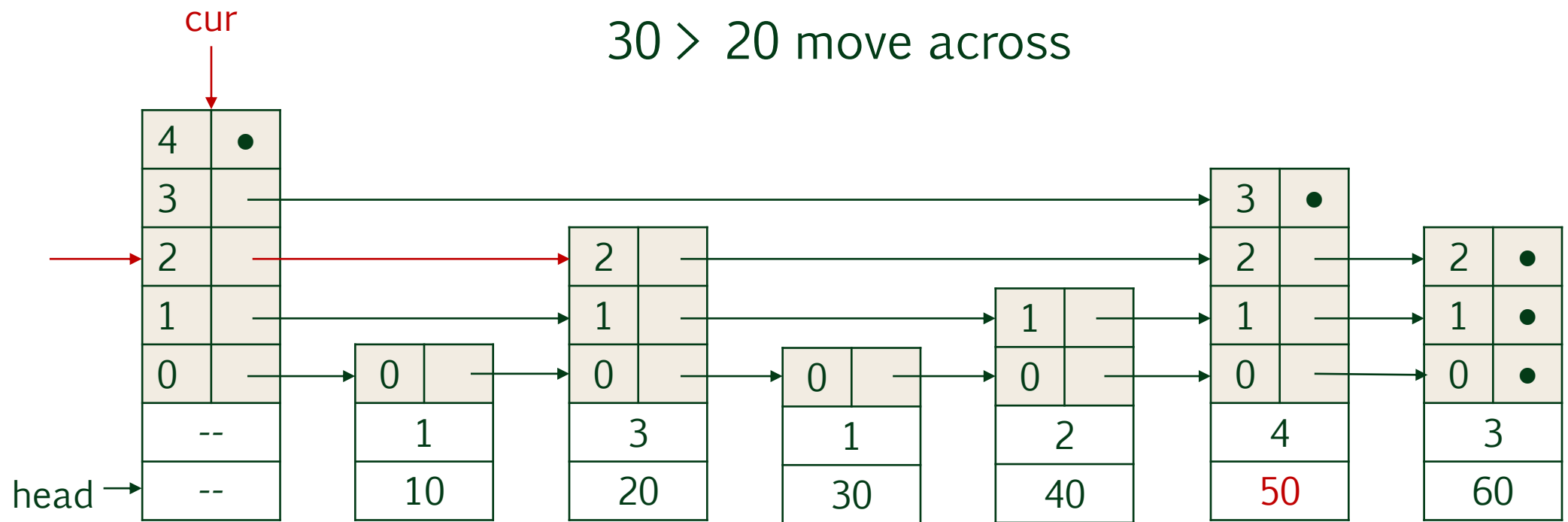


Contains 30





Contains 30

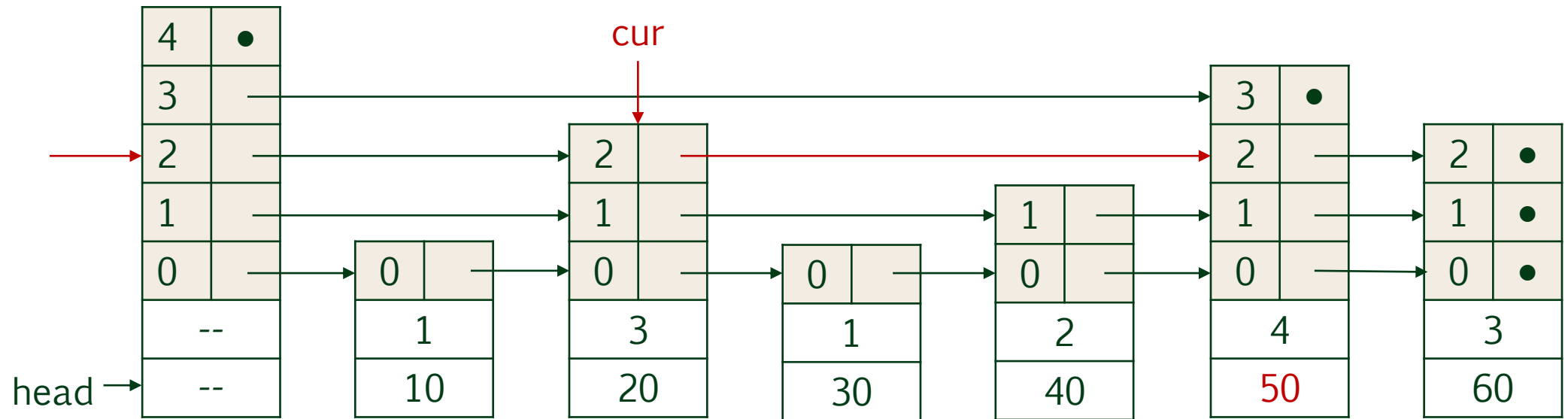


maxHeight = 4



Contains 30

$30 < 50$ move down



maxHeight = 4

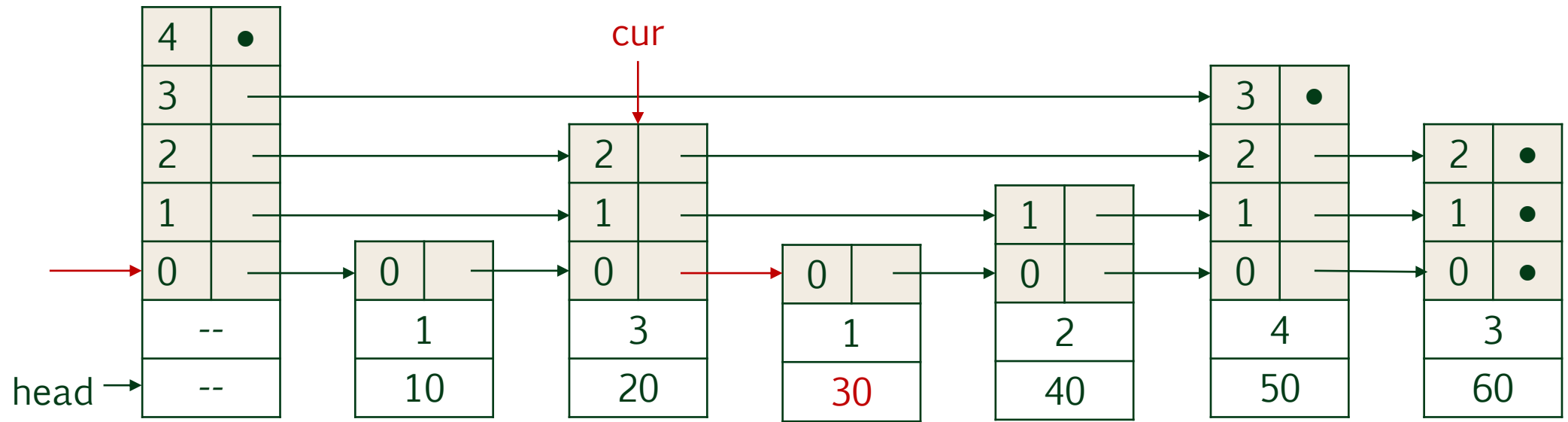


The diagram illustrates a linked list structure with 6 nodes. The first node is a header node with indices 0 to 4 and a null pointer. It points to a node with value 10. The second node points to a node with value 20. The third node points to a node with value 30. The fourth node points to a node with value 40. The fifth node points to a node with value 50. The sixth node points to a node with value 60. A red arrow labeled 'cur' points to the second node (value 20). A red arrow points to the first node's index 1 cell. A red arrow points to the fourth node's value 40.

maxHeight = 4



Contains 30



Found!

maxHeight = 4



Exercises (using the Final Skip List)

- › Show the sequence of references that are followed to find:
 - 60
 - 30
 - 10
 - 70
 - 45
 - 35

- › Draw two instances of the skip list where the Contains method takes linear time (i.e., $O(n)$ steps).



Rules of thumb

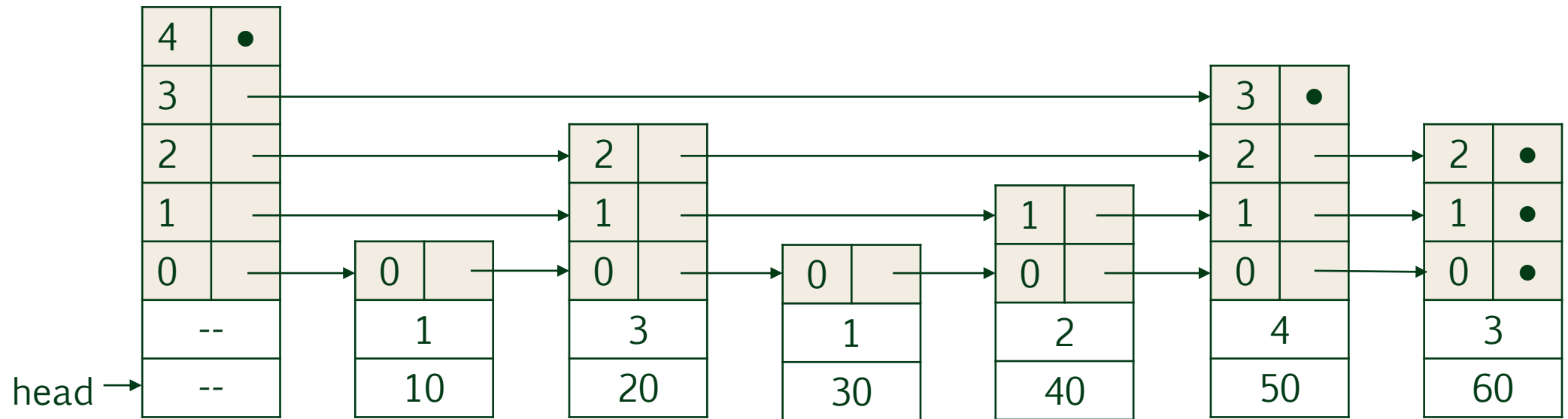
- › Drop down a level when the item you wish to insert, find, or remove is **less than** `cur.Next[i].Item`
 - In the case of Insert, references are adjusted first when the height of the new node is within range
- › Move across a level when the item you wish to insert, find or remove is **greater than** `cur.Next[i].Item`



- › When the item is **equal to** `curr.Next[i].Item`:
 - Insert: References are adjusted first when the height of the new node is within range (i.e. duplicate items)
 - Contains: Return true
 - Remove: `cur.Next[i] = cur.Next[i].Next[i]`



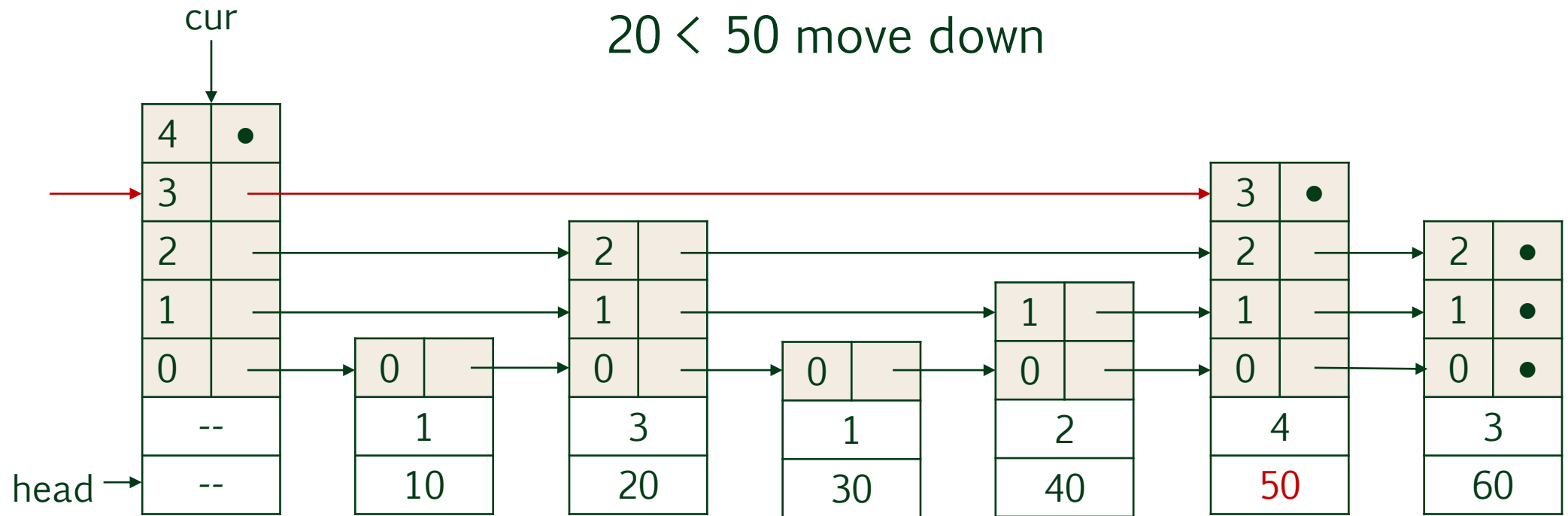
Starting with our final skip list



maxHeight = 4



Remove 20

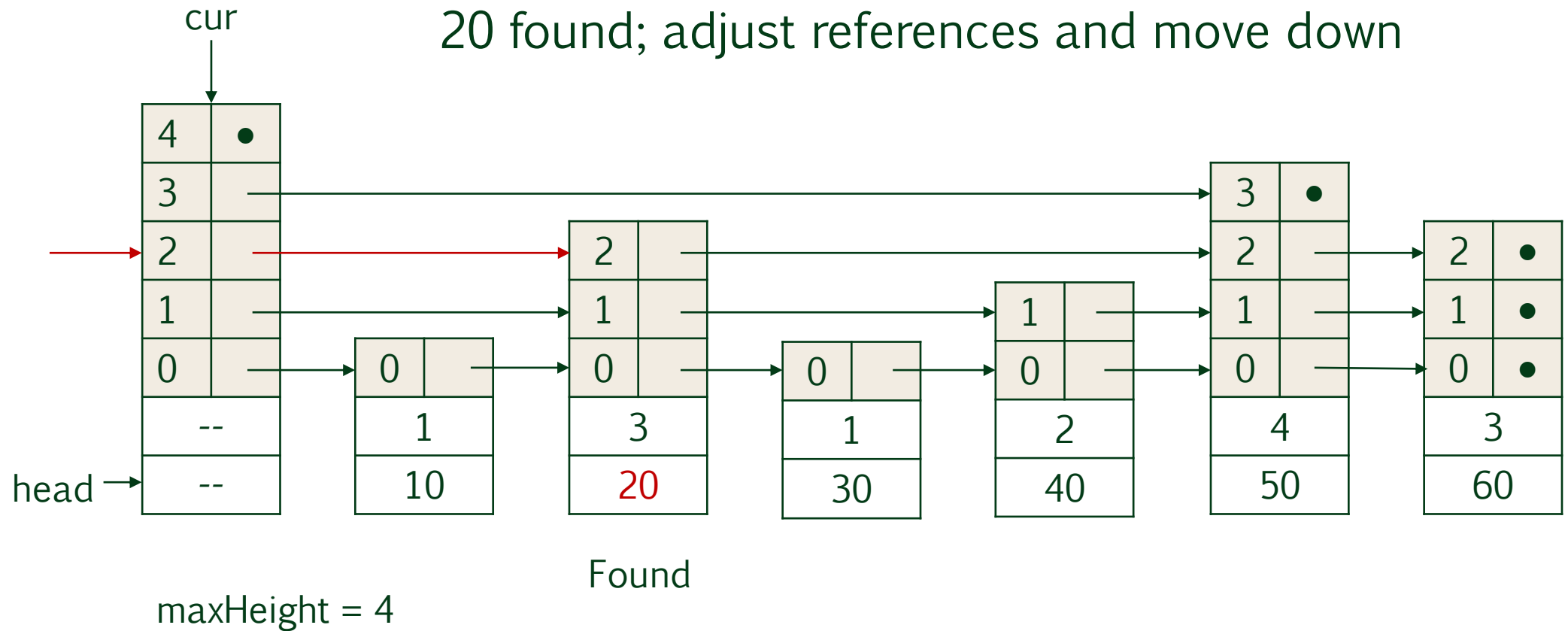


maxHeight = 4



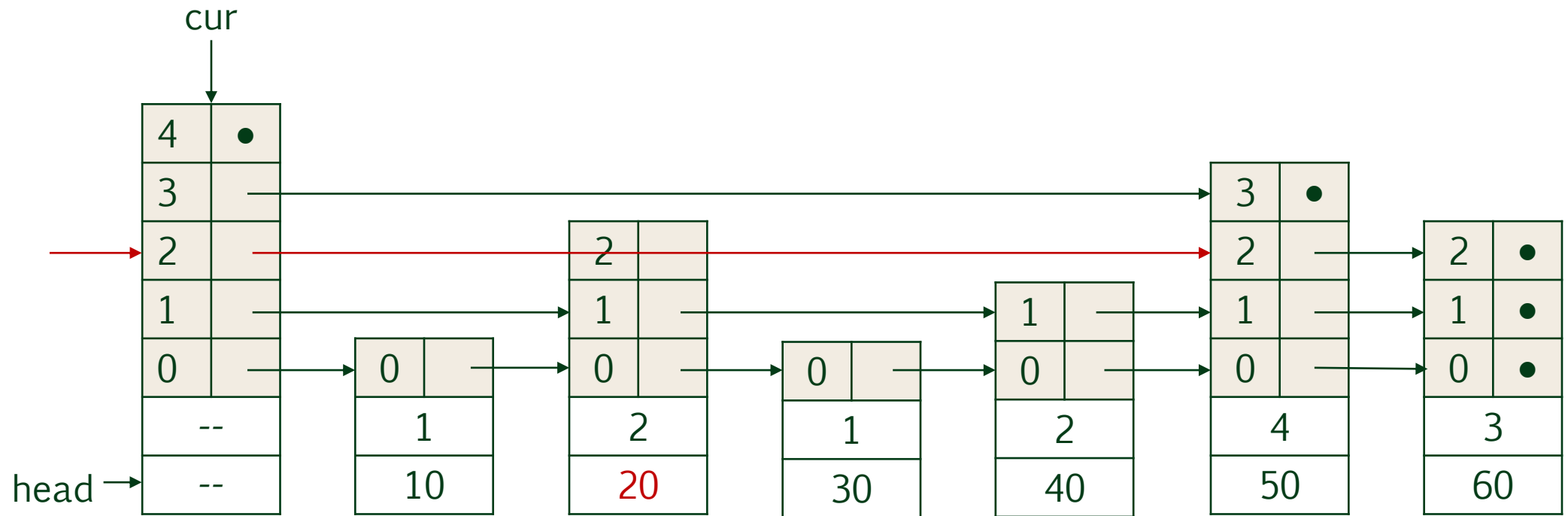
Remove 20

20 found; adjust references and move down





Remove 20

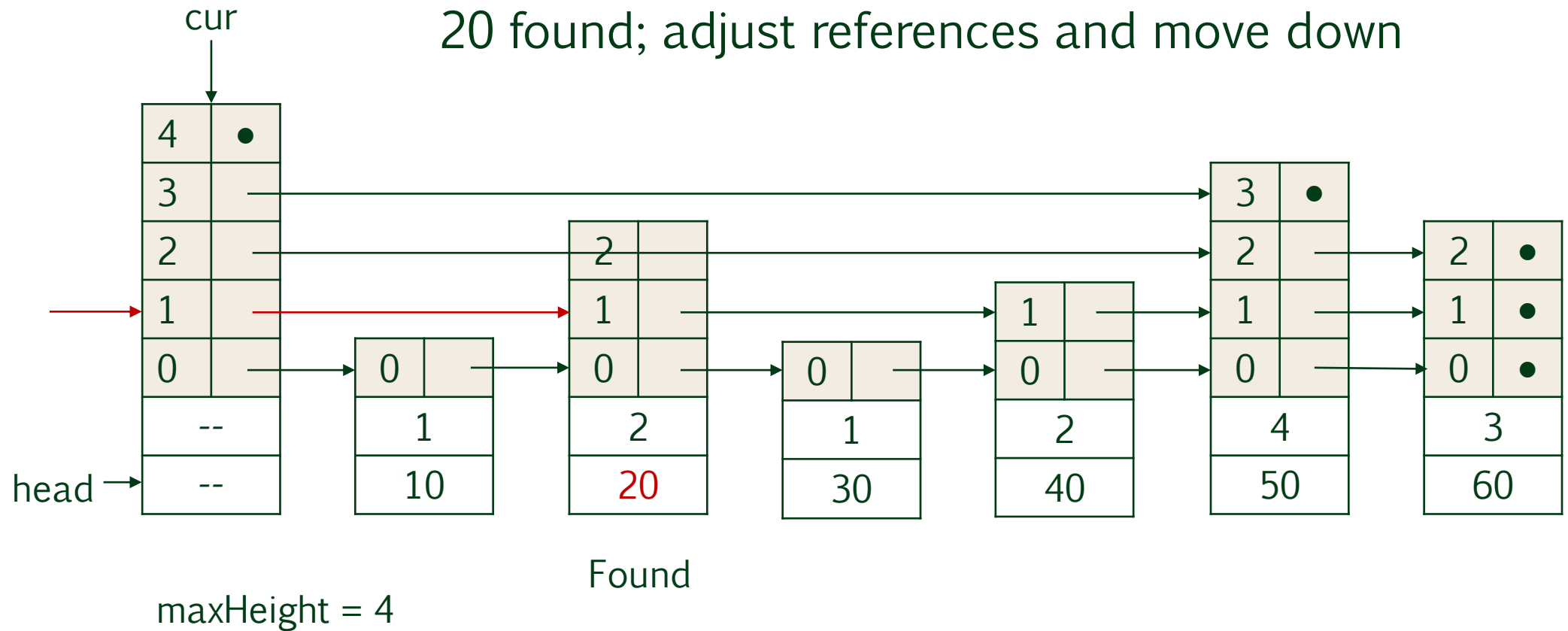


maxHeight = 4



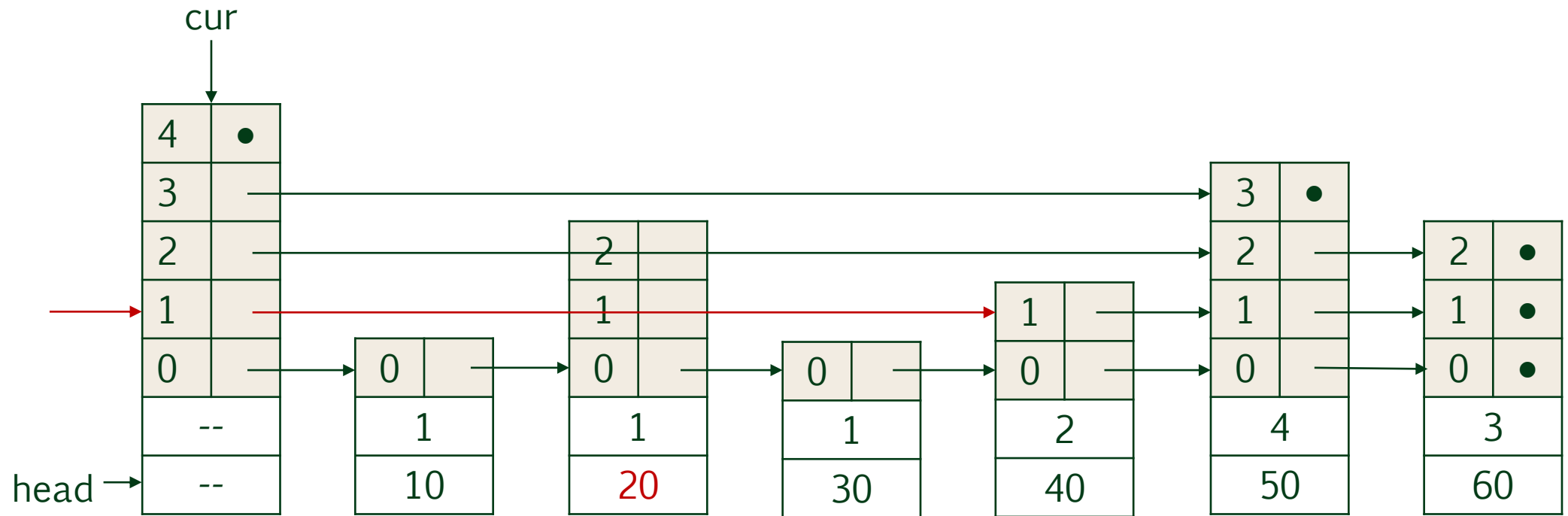
Remove 20

20 found; adjust references and move down





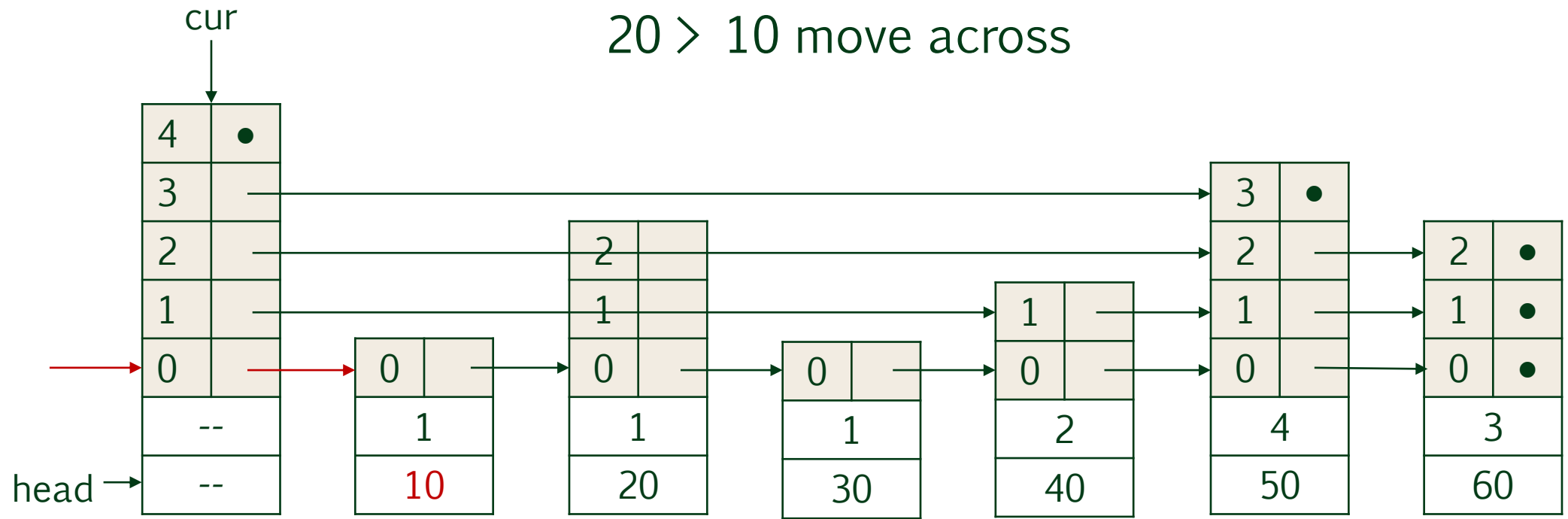
Remove 20



maxHeight = 4



Remove 20

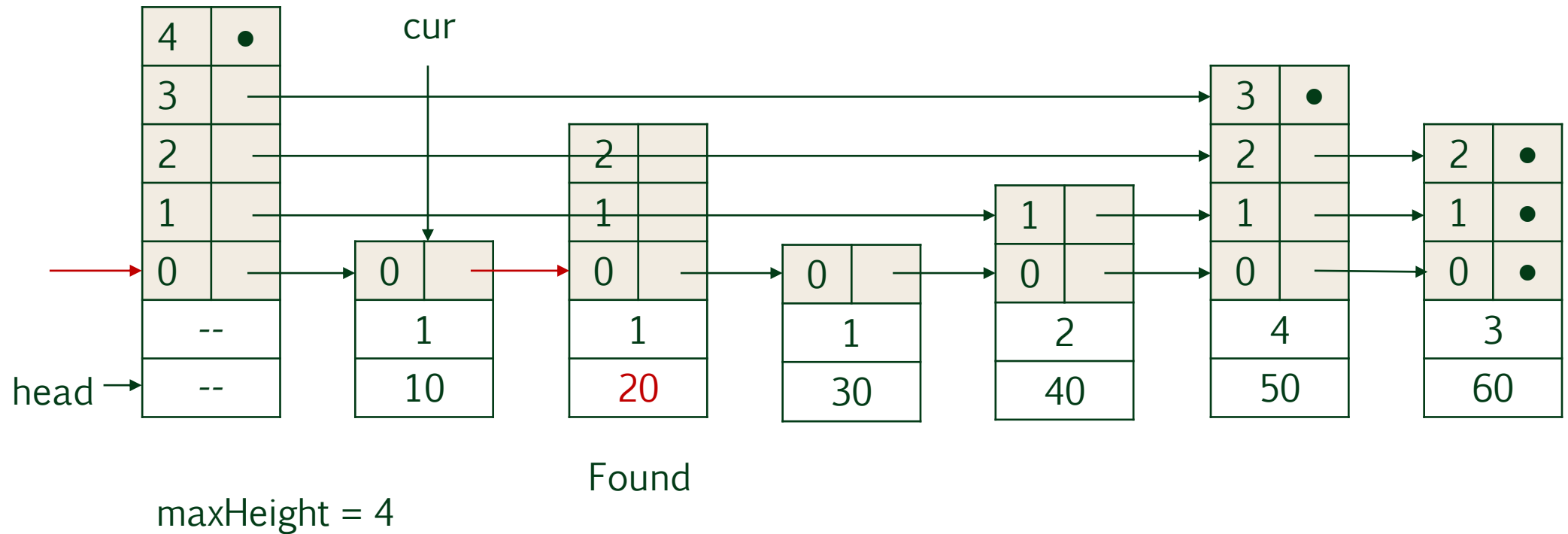


maxHeight = 4



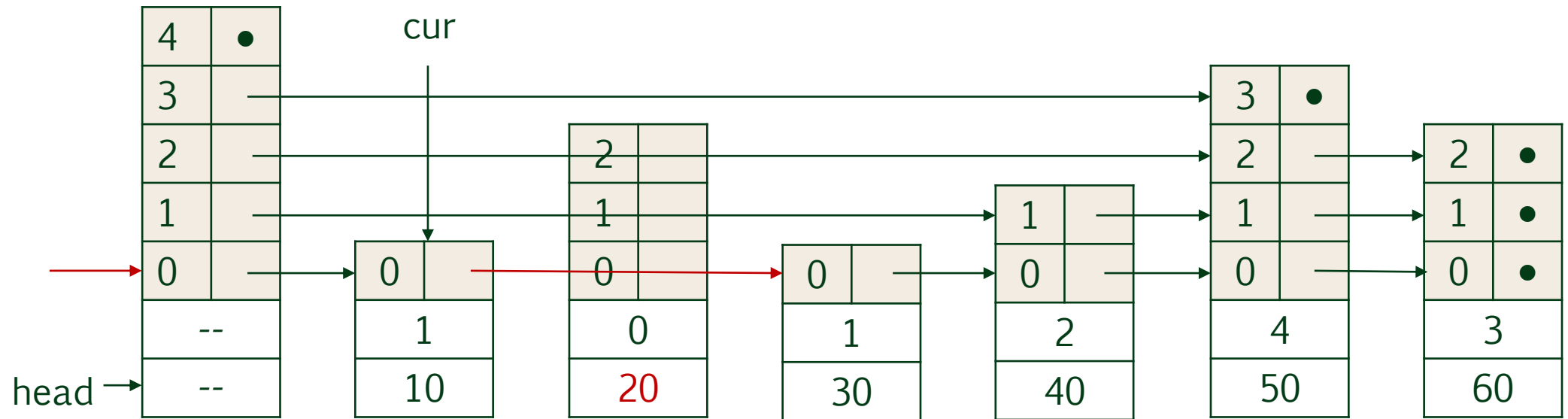
Remove 20

20 found; adjust references and move down





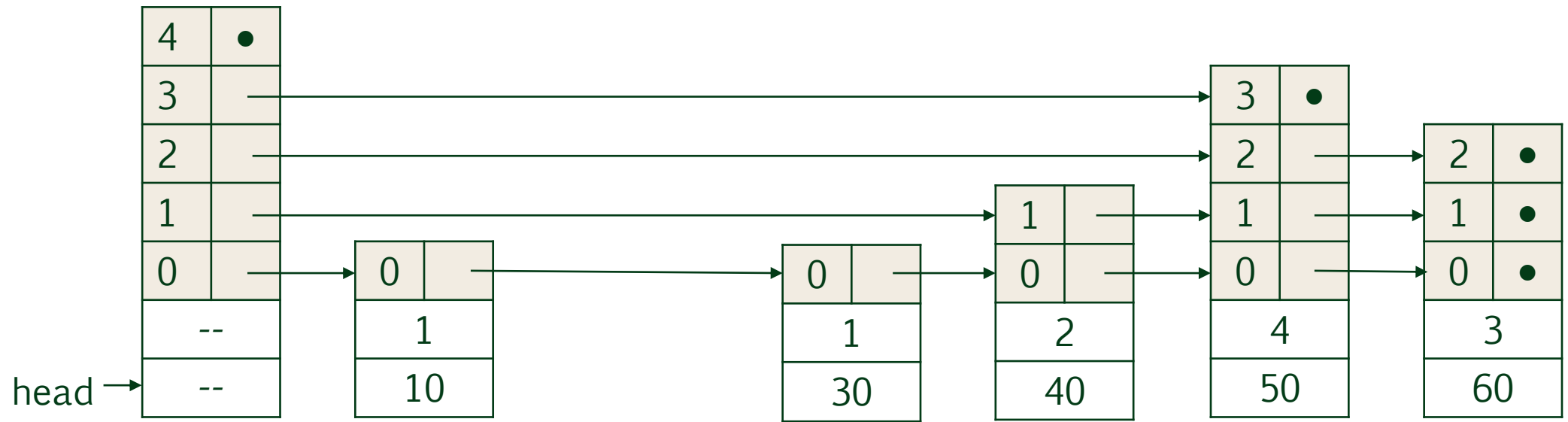
Remove 20



maxHeight = 4



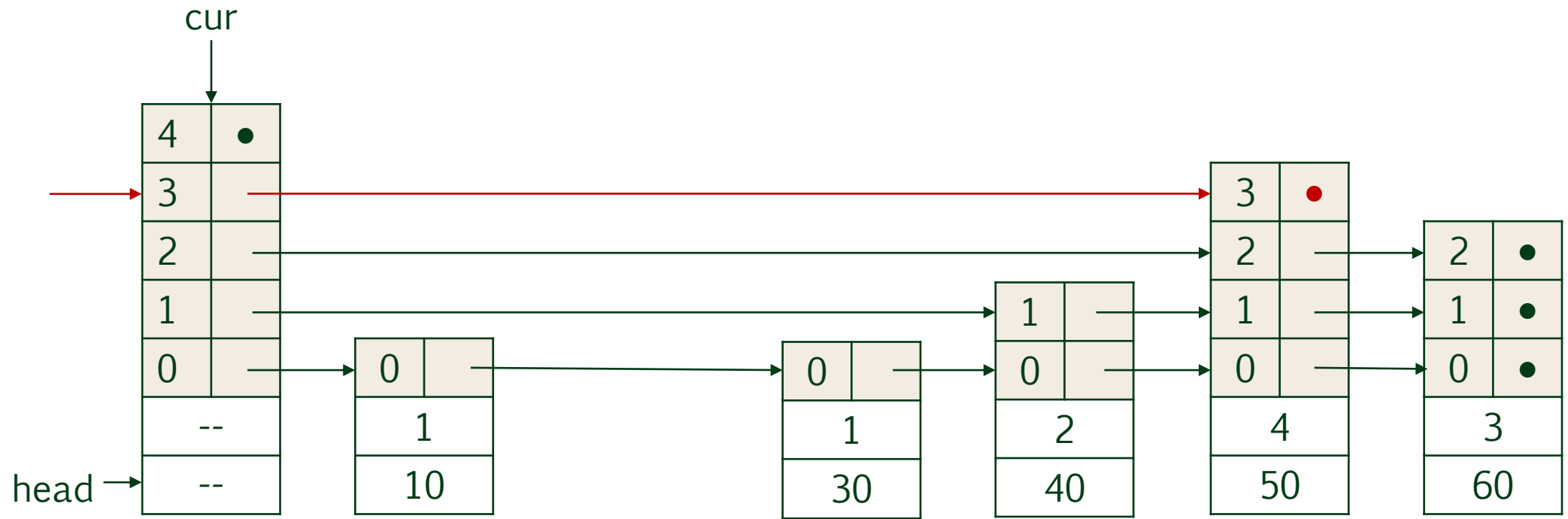
Remove 20



maxHeight = 4



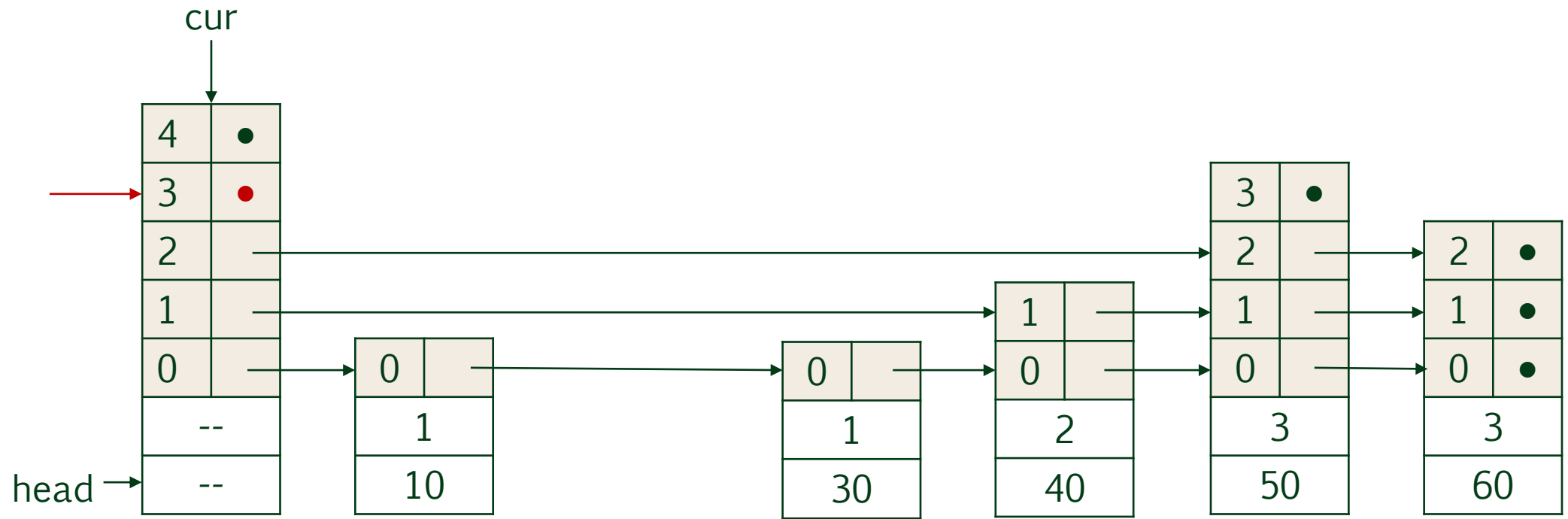
Remove 50



maxHeight = 4



Remove 50

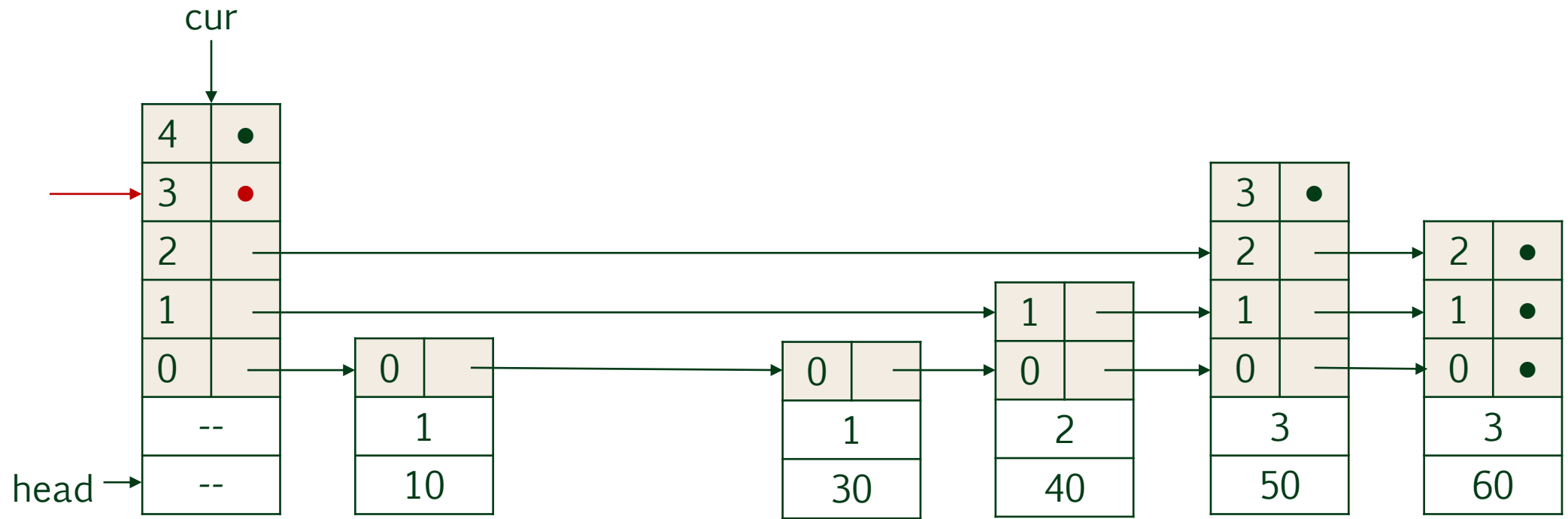


maxHeight = 3

Note: maxHeight decreases by 1 whenever head.Next[i] is reset to null



Remove 50

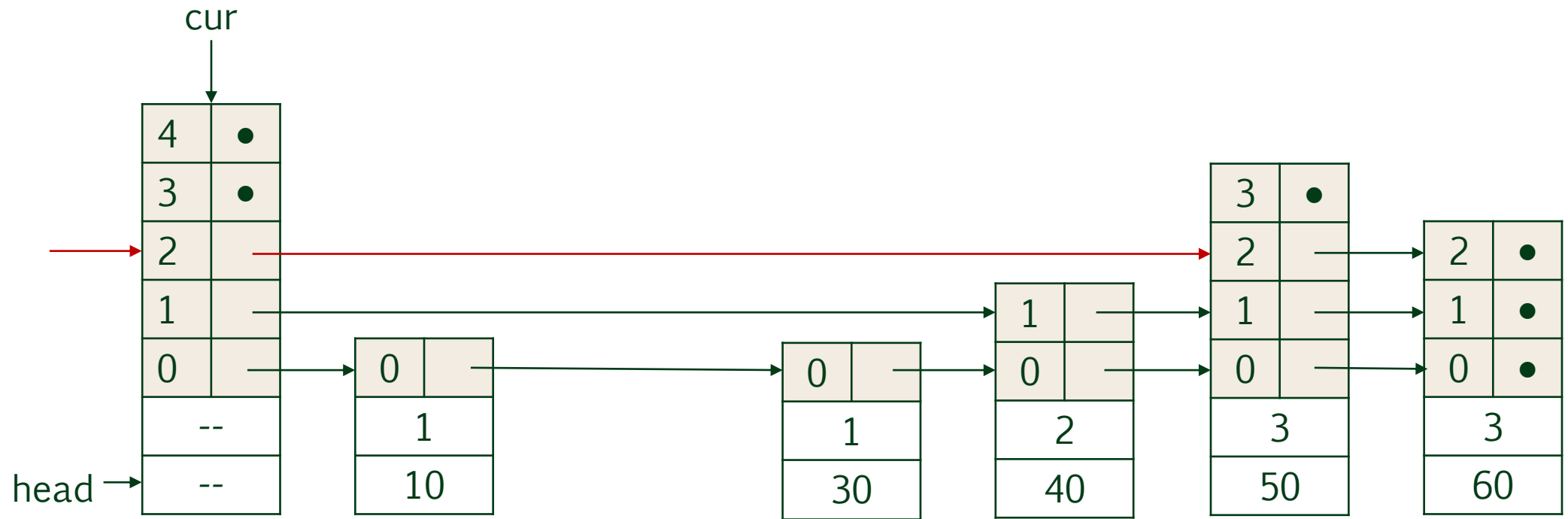


maxHeight = 3

Note: maxHeight decreases by 1 whenever head.Next[i] is reset to null



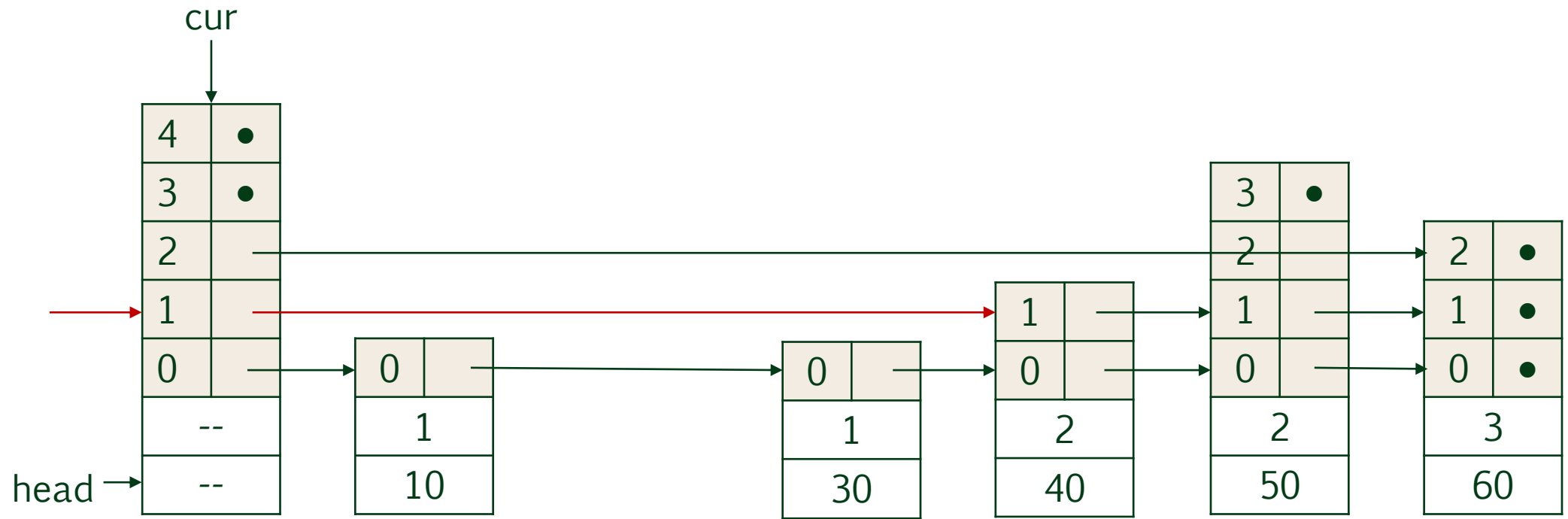
Remove 50



maxHeight = 3



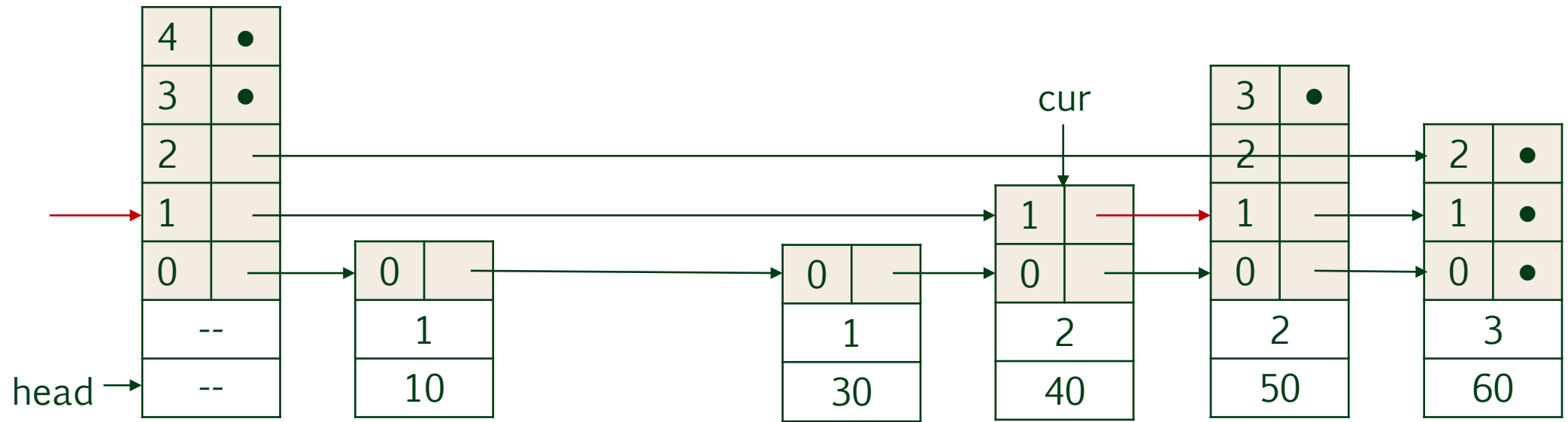
Remove 50



maxHeight = 3



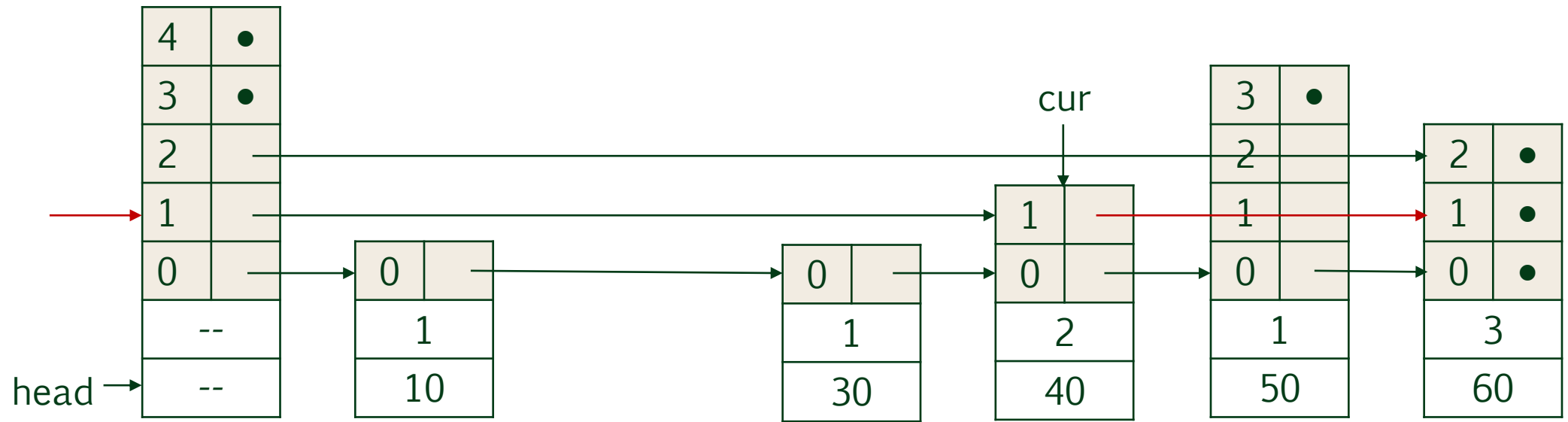
Remove 50



maxHeight = 4



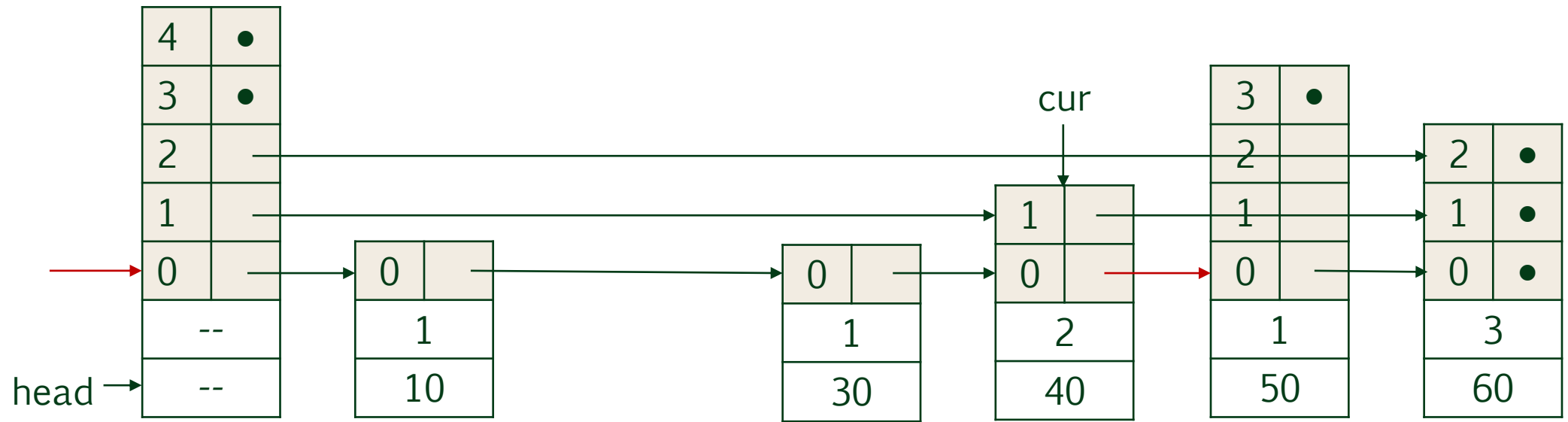
Remove 50



maxHeight = 3



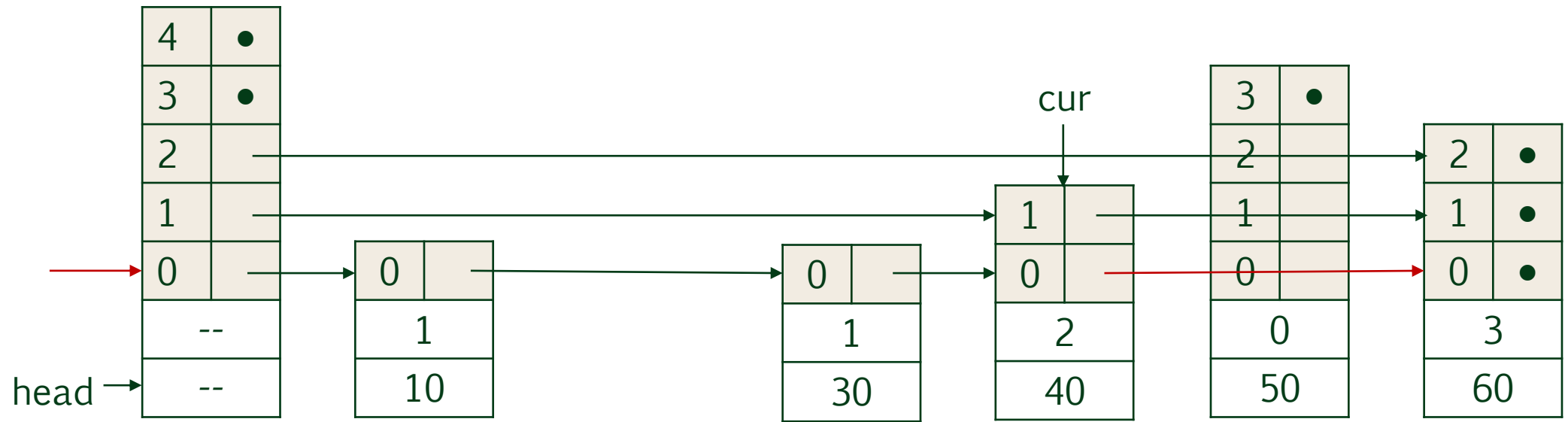
Remove 50



maxHeight = 3



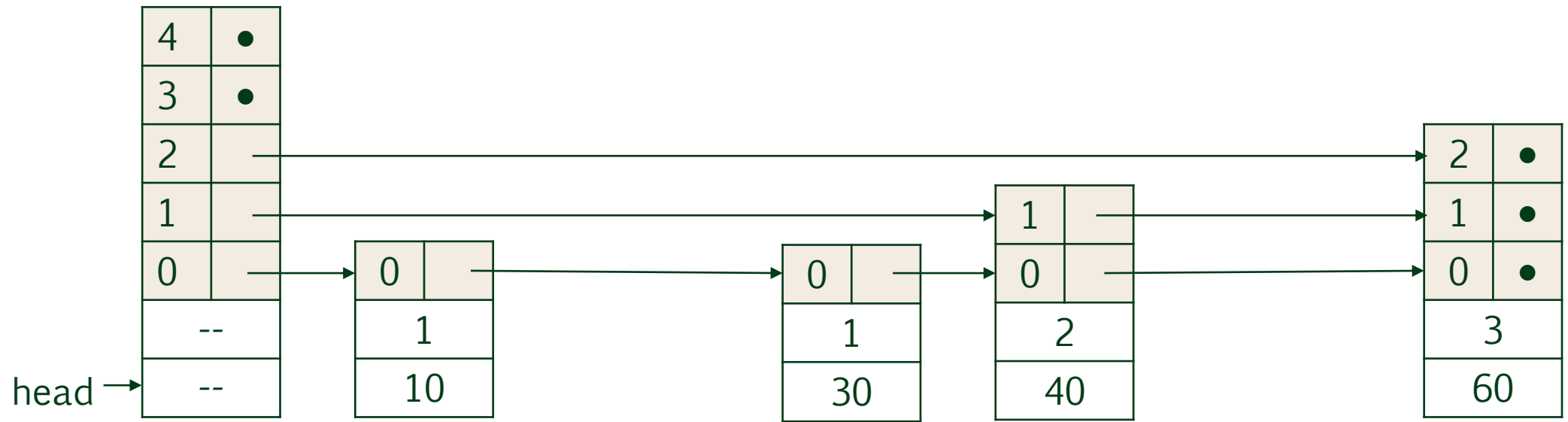
Remove 50



maxHeight = 3



Remove 50



maxHeight = 3

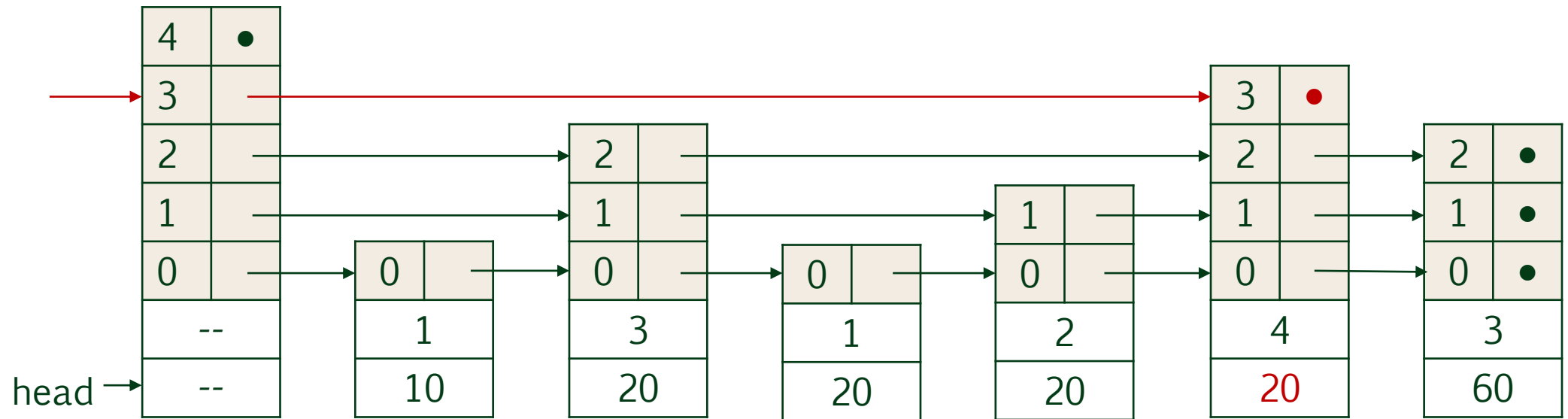


Removing a duplicated item

- › To remove **one** instance of a duplicate item from the skip list, some interesting and subtle issues arise
- › Consider the following skip list where the item 20 is duplicated 4 times



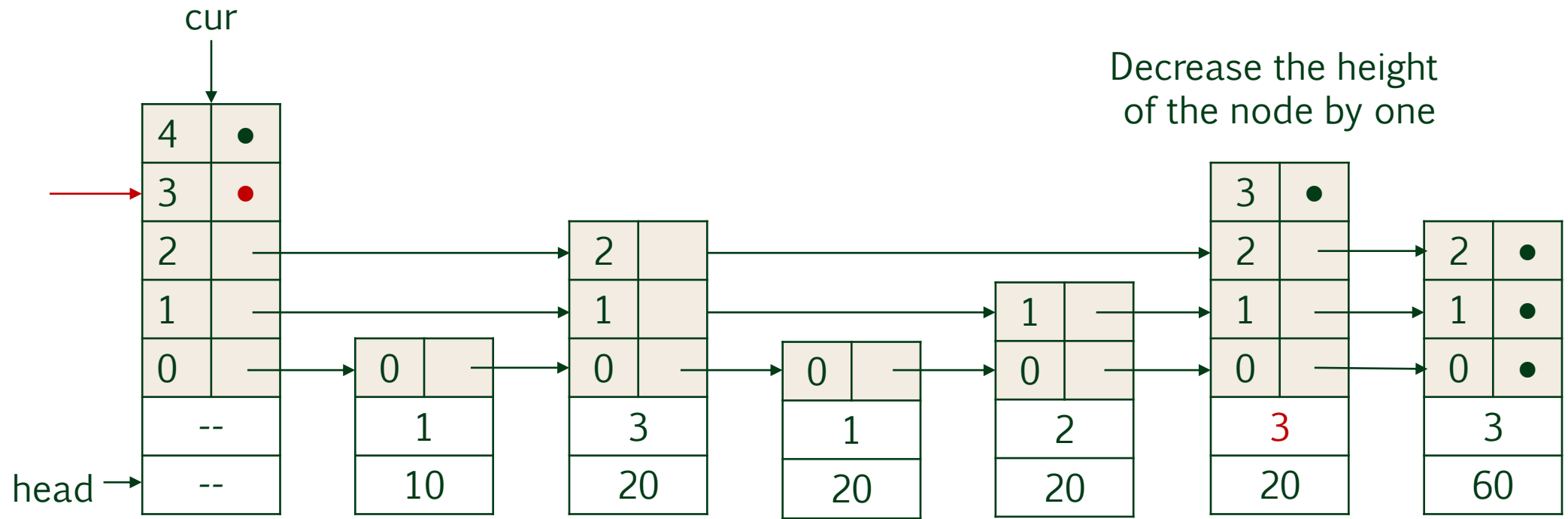
To remove one instance of 20



maxHeight = 4



Remove one instance of 20

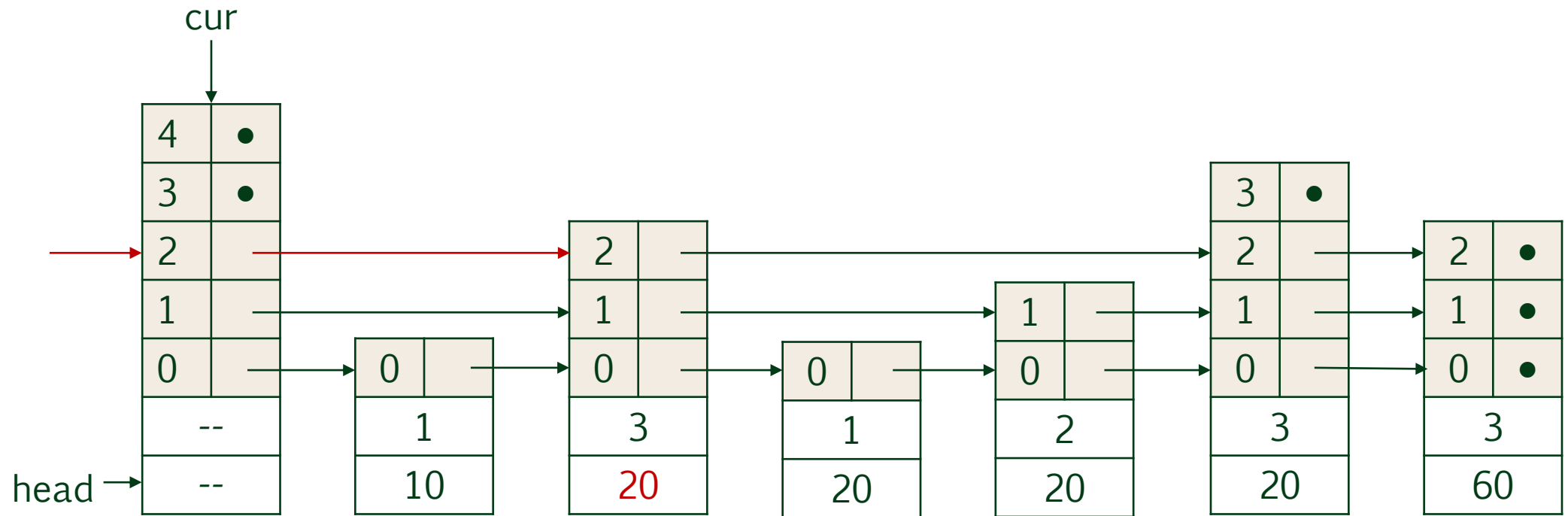


maxHeight = 3

Decrease the maxHeight by one
since `cur.Next[3] == null`



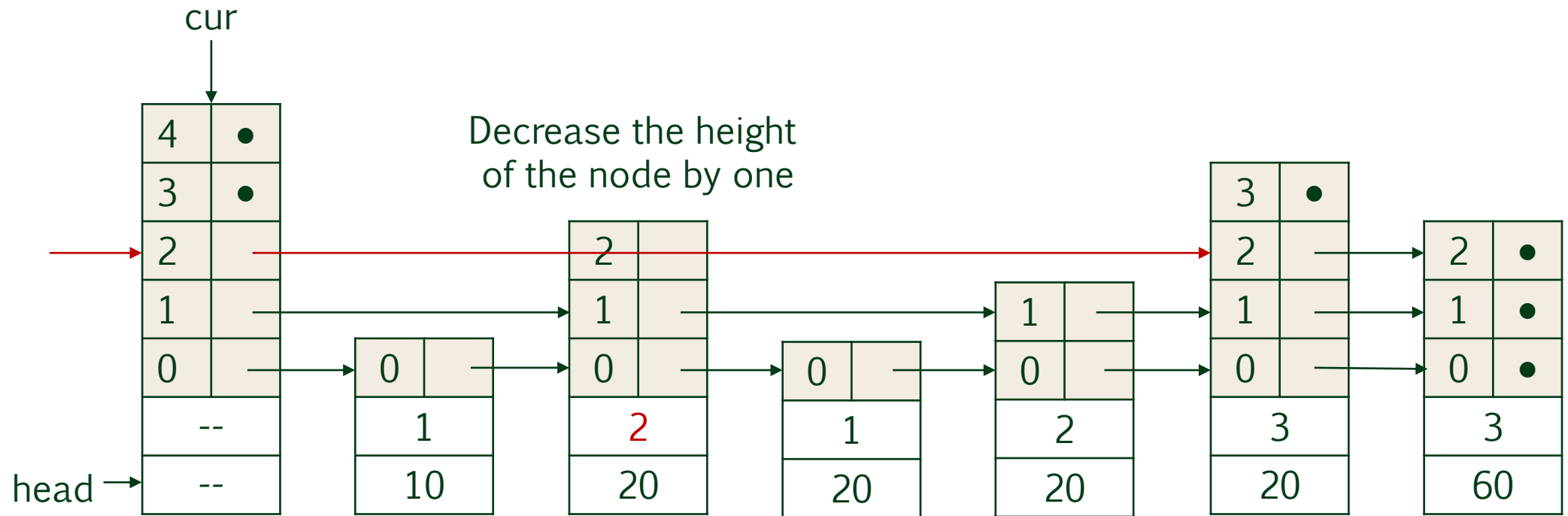
Remove one instance of 20



maxHeight = 3



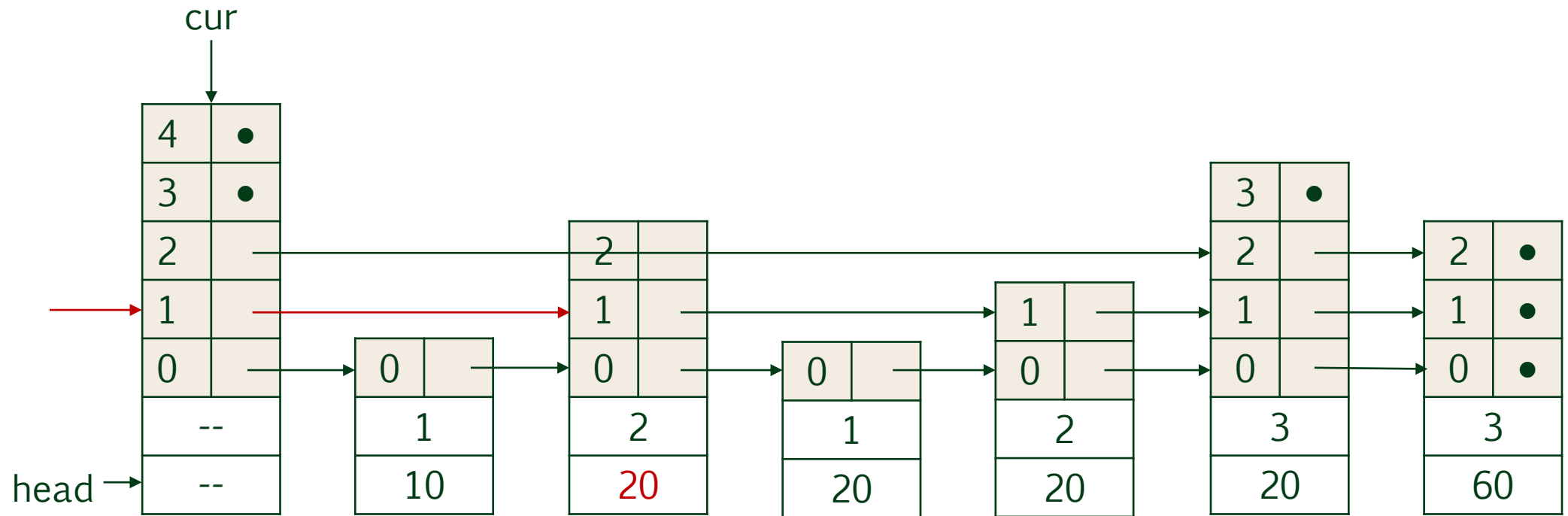
Remove one instance of 20



maxHeight = 3



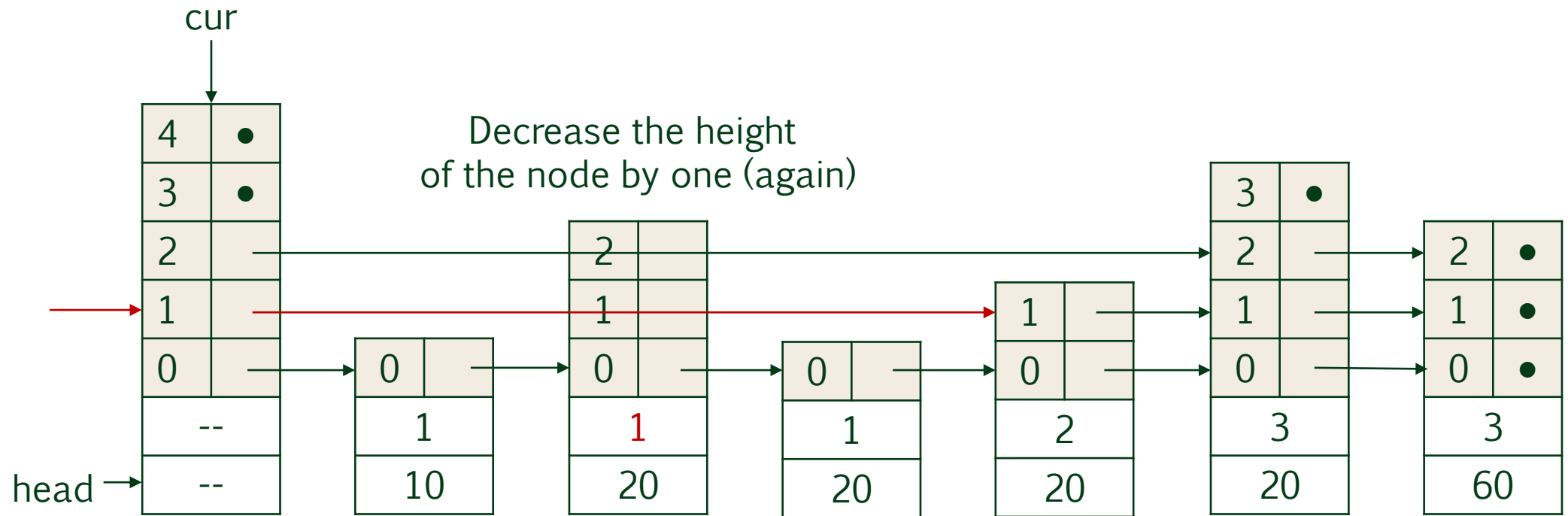
Remove one instance of 20



maxHeight = 3

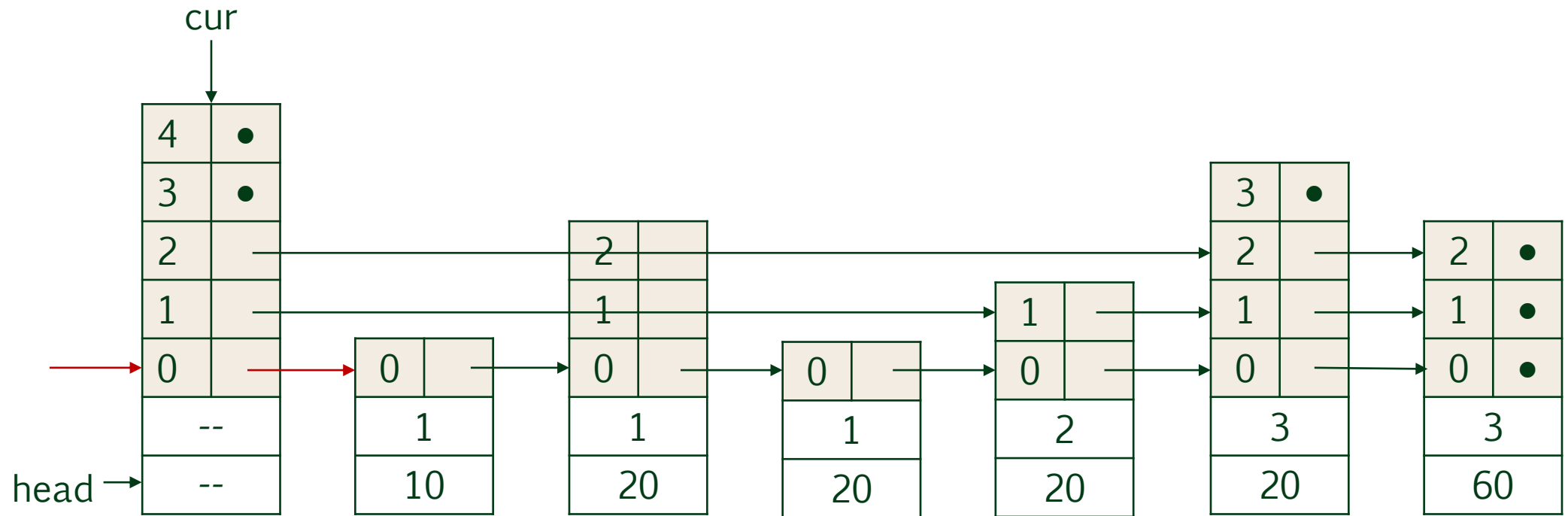


Remove one instance of 20





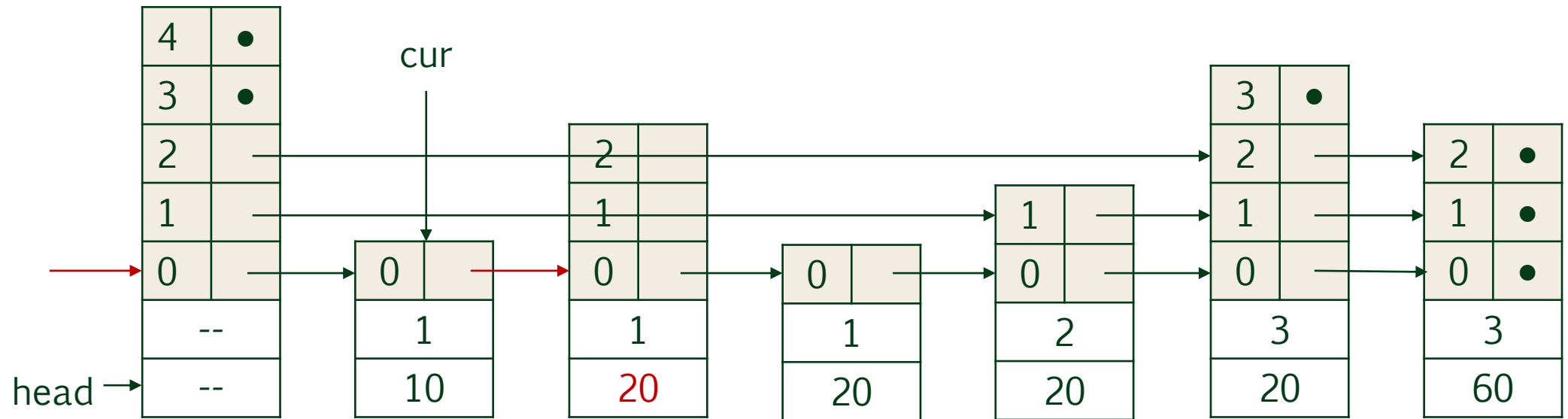
Remove one instance of 20



maxHeight = 3



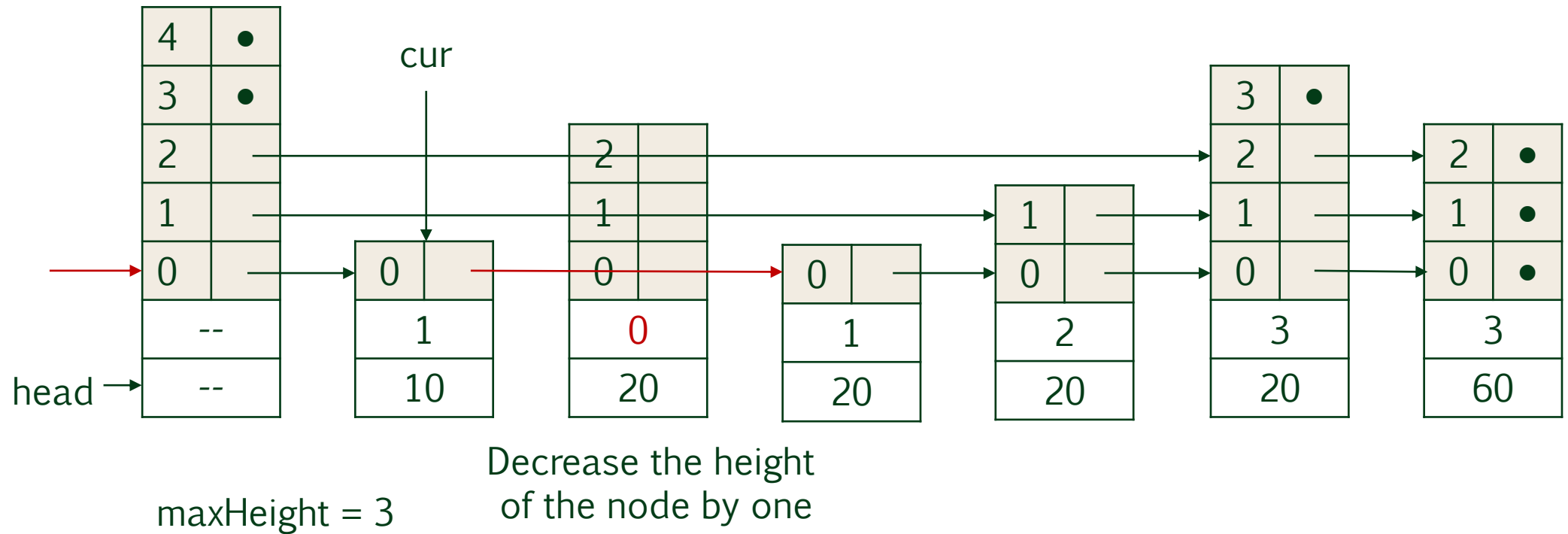
Remove one instance of 20



maxHeight = 3

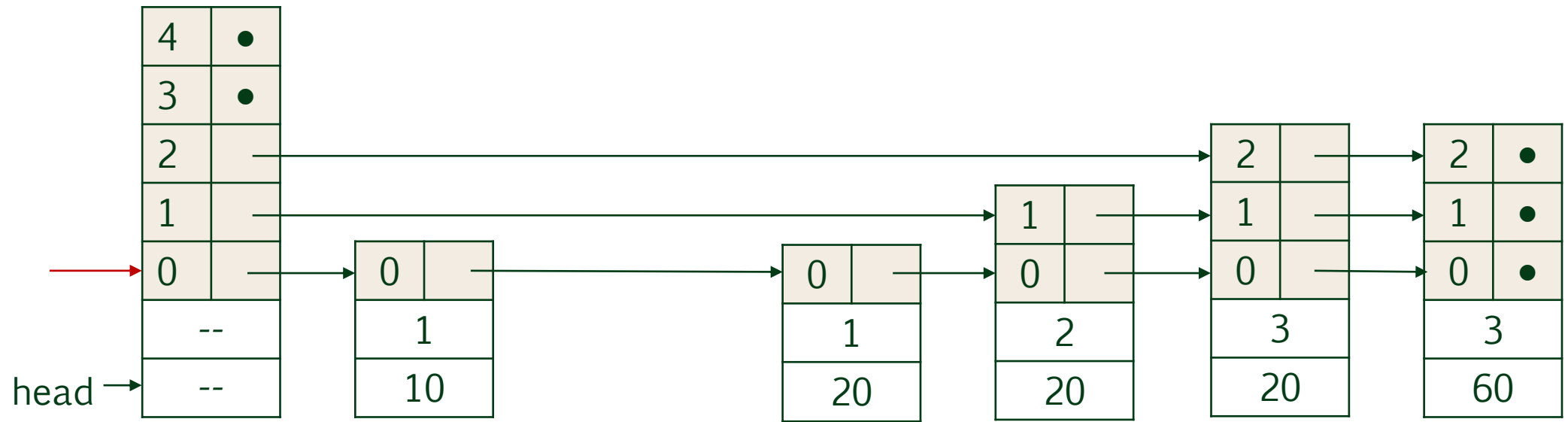


Remove one instance of 20





Skip list after removing one instance of 20





Exercises (using the Final Skip List)

- › Remove 60, 10, and 45.
- › If $\text{maxHeight} = 1$ then show that Remove behaves exactly as the Remove method of a singly-linked list.
- › If an item is duplicated in nodes of
 - increasing height
 - decreasing heightshow how one instance of the item is removed.



How to determine the random height?

› Strategy:

- Generate a positive random 32-bit integer R
- Let the height be the number of consecutive lower order bits that are equal to one
- Set the height of a node equal to $\min(\text{height}+1, \text{maxHeight}+1)$



Example

R	0	1	0	0	0	0	1	1	1	0	0	0	1	0	1	0	1	1	1	0	1	1	1	0	0	0	0	0	0	1	1	1
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	
R & 1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	

height = 1; R>>1 // right shift one place

R 1 R & 1	0	0	1	0	0	0	0	1	1	1	0	0	0	1	0	1	0	1	1	1	0	1	1	1	0	0	0	0	0	0	1	1
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	

height = 2; R>>1 // right shift one place

R	0	0	0	1	0	0	0	0	1	1	1	0	0	0	1	0	1	0	1	1	1	0	1	1	1	0	0	0	0	0	0	1
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	
R & 1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	

height = 3; R>>1 // right shift one place



R
1
R & 1

0	0	0	0	1	0	0	0	0	1	1	1	0	0	0	1	0	1	0	1	1	1	0	1	1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Loop ends when $R \& 1 == 0$

Height of the new node is set at 4



Implementation

```
int height = 0;
int R = rand.Next();    // R is a random 32-bit positive integer
while ((height < maxHeight) && ((R & 1) == 1))
{
    height++;    // Equal to the number consecutive lower order 1-bits
    R >>= 1;    // Right shift one bit
}
if (height == maxHeight) maxHeight++;

Node newNode = new Node(item, height + 1);
```



* Expected time complexity

› Determined by the following loop structure

```
for (int i = maxHeight - 1; i >= 0; i--) // Down
{
    while (cur.Next[i] != null && cur.Next[i].Item.CompareTo(item) < 0) // Across
        cur = cur.Next[i];
    ...
}
```



- › Expected height of the skip list is $O(\log n)$
 - The for loop is expected to iterate $O(\log n)$ times
- › The number of nodes at level $i+1$ is expected to be half the number of nodes at level i . Therefore, for each link at level $i+1$, there are an expected 2 links at level i .
 - The while loop is expected to iterate $O(1)$ times
- › The overall expected time complexity is $O(\log n)$ for Insert, Contains, and Remove since all three methods use the loop structure on the previous slide